

RESEARCH

Open Access



Hybrid dolphin swarm sparrow search optimization based multi-objective cloud workflow scheduling

S. Subashree^{1*}, M. Rajakumaran², G. Pushpa¹ and S. Senthilkumar³

Abstract

In cloud computing, scheduling workflows for data-intensive tasks is challenging due to task dependencies, heterogeneous resources, and high computational demands, all of which affect cost, execution time, and energy usage. This research proposes a novel hybrid optimization algorithm called Dolphin Swarm Sparrow Search Optimization (DSSSO) to address these challenges. The DSSSO model combines the broad search capability of Dolphin Swarm Optimization with the precise tuning ability of Sparrow Search Algorithm, enabling effective exploration and exploitation in the solution space. Unlike existing scheduling models that primarily optimize makespan and cost, the proposed DSSSO explicitly integrates energy consumption and resource utilization into a unified multi-objective framework. This dual-phase design—leveraging dolphins for global exploration and sparrows for local exploitation—ensures both convergence stability and adaptability in heterogeneous cloud environments. The main objective is to minimize makespan, reduce energy consumption, and lower scheduling cost while maximizing resource utilization. The model is evaluated using benchmark scientific workflows such as CyberShake, Montage, Epigenomics, and LIGO. Simulation results show that DSSSO outperforms existing methods, achieving up to 12% lower makespan, 15% less energy consumption, and 10% better resource usage compared to algorithms like Hybrid Bat Optimization and Improved Bat Optimization.

Keywords Cloud computing, Workflow scheduling, Optimization algorithms, Dolphin swarm optimization, Sparrow search algorithm

Introduction

The innovative cloud computing technology allows the users to perform different operations with large amount of data through the network of servers deployed in wide remote areas. Large number of organizations and Government entities now rely on cloud computing and its services. Cloud offers a wide range of solutions to everyone's needs in the present era. Cloud services for business and scientific applications are mainly task oriented and its workflows structure the business needs [1]. The scientific workflows are mainly data dependent, and it has highly intensive applications. These applications are structured into scientific works and managed through

*Correspondence:

S. Subashree
drsubashree1989@gmail.com

¹Department of Computer Science and Engineering, E.G.S. Pillay Engineering College, Nagapattinam, Tamil Nadu 611002, India

²School of Computing, SASTRA Deemed University, Thanjavur, Tamil Nadu 613401, India

³Department of Electronics and Communication Engineering, E.G.S. Pillay Engineering College, Nagapattinam, Tamil Nadu 611002, India

scheduling systems. However, the major issue in scientific workflow occurs when scheduling large workflow tasks with scientific standards [2]. Moreover, the adaptation of heterogeneous cloud computing added to the challenges and complexities in scientific workflow task scheduling. When it comes to quality of service again large workflow scheduling should provide reliability, scalability and resource utilization to users which makes this scheduling process as NP-hard problem. Thus, it is essential to introduce solutions which should provide better Quality of Services (QoS) to users.

The research community provided a wide range of solutions to manage and schedule the scientific workflow applications. The existing researches considered user defined QoS parameters in addition to communication rates and processing capacity. However, the major challenge of workflow scheduling is present in minimizing the cost, make span and energy consumption while accelerating the throughput. It is essential to consider these multiple constraints while optimizing the resources. Moreover, different workflow applications require various strategies when they are adapted to heterogeneous systems. The review presented in [3] provides a comprehensive summary of user tasks and workflow scheduling techniques based on architectures. Another review presented in [4] summarizes different workflow applications and scheduling techniques under different categories like resource scheduling, energy efficient scheduling and fault tolerant scheduling. Both review works summarize that optimization algorithms will be the best choice for scheduling scientific workflow applications.

Optimization algorithms are developed to provide optimal solutions to real world optimization. The naturally occurred events are formulated and created as optimization algorithms. Heuristic, nature inspired, metaheuristic and evolutionary optimization algorithms are developed and utilized in various domains like image processing, networking, signal processing, etc. Various optimization algorithms are used for workflow applications [5, 6]. Different types of metaheuristic optimization algorithms are studied in [7] for workflow scheduling and provided a detailed comparative analysis which is quite useful to summarize the limitations of conventional optimization algorithm-based workflow scheduling. Another review presented in [8] considered PSO algorithm and its merits and demerits in scheduling in detail. Researchers developed workflow scheduling based on optimization algorithms like genetic algorithm, bat optimization algorithm, ant colony optimization [9–12], whale optimization [13]. However, the existing approaches mainly considered make span and cost as their objective functions. It is essential to consider other metrics like energy consumption and resource utilization in objective functions to provide better QoS to users. While previous studies have

explored various optimization-based workflow scheduling techniques, most focus primarily on minimizing makespan and cost, often overlooking equally critical factors such as energy consumption and resource utilization. Moreover, many existing approaches struggle with maintaining a balance between exploration and exploitation during the optimization process, frequently resulting in premature convergence and suboptimal task-resource mappings. These limitations highlight a clear research gap: the need for a robust, multi-objective scheduling algorithm that can effectively address multiple QoS constraints in heterogeneous cloud environments while maintaining solution diversity and convergence stability.

To bridge this gap, this research introduces a novel hybrid optimization algorithm—Dolphin Swarm Sparrow Search Optimization (DSSSO)—that integrates the global exploration capabilities of Dolphin Swarm Optimization (DSO) with the local refinement efficiency of the Sparrow Search Algorithm (SSA). This hybrid model is specifically designed to optimize four critical objectives simultaneously: makespan, energy consumption, cost, and resource utilization. By leveraging the complementary strengths of DSO and SSA, the DSSSO algorithm offers a more adaptive and scalable solution for workflow scheduling compared to conventional and hybrid metaheuristic models.

The proposed DSSSO framework stands out by addressing multiple shortcomings in the literature: it incorporates energy and resource-aware objectives often ignored in prior work, employs a dual-phase optimization to avoid local optima, and demonstrates superior performance across diverse scientific workflows. This clear enhancement over existing techniques establishes the novelty and practical relevance of the research.

Most existing workflow scheduling approaches optimize only one or two objectives, typically makespan and cost, while critical aspects such as energy consumption and resource utilization remain underexplored. Moreover, conventional optimization models frequently suffer from premature convergence due to imbalance between exploration and exploitation, leading to suboptimal task-to-resource mappings in heterogeneous cloud settings. This leaves a clear research gap: the need for a unified, multi-objective scheduling model that simultaneously addresses makespan, cost, energy consumption, and resource utilization, while ensuring convergence stability and scalability in dynamic cloud environments. The present work is motivated by this gap and introduces the DSSSO framework as a robust solution.

Thus, a hybrid optimization is presented in this research work as a multi-objective model which provides optimal solution for workflow scheduling application. The research work major contributions are summarized as follows.

- This research presents a novel hybrid optimization algorithm that combines Dolphin Swarm Optimization (DSO) and Sparrow Search Algorithm (SSA) for multi-objective workflow scheduling problem in cloud computing environments. The hybrid model utilizes the strengths of both algorithms to optimize multiple objectives such as makespan, cost, energy consumption, and resource utilization.
- The proposed method is formulated to handle multiple objectives through the hybrid DSSSO algorithm and compared to existing models that typically focus on one or two objectives. By integrating DSO's strong exploration capabilities with SSA's efficient exploitation abilities, the hybrid DSSSO algorithm achieves a better balance between exploring the solution space and refining solutions. Also, the proposed DSSSO helps to avoid local optima and improves convergence.
- The research provides detailed experimental validation of the proposed DSSSO model using various benchmark scientific workflows such as CyberShake, Montage, Epigenomics, and LIGO. These experiments demonstrate the effectiveness of the hybrid model in reducing makespan, minimizing costs, and improving resource utilization compared to other state-of-the-art optimization methods.
- A thorough comparative analysis is presented with existing optimization methods, such as Particle Swarm Optimization Algorithm (PSOA), Ant Colony Optimization Algorithm (ACOA), Whale Optimization Algorithm (WOA), Hybrid Bat Optimization Algorithm (HBOA) and Improved Bat Optimization Algorithm (IBOA). The results show that the proposed hybrid DSSSO outperforms these methods in cloud workflow scheduling tasks.

The rest of the discussions are presented in the following order. “[Related work](#)” section presents a detailed literature review on existing workflow scheduling models. The proposed hybrid optimization-based workflow scheduling is presented in “[Proposed work](#)” section. “[Results and discussion](#)” section presents the results and discussions and “[Conclusion](#)” section presents the conclusion of research work.

Related work

Optimization algorithm used for workflow scheduling are considered for research analysis. The existing methodology and its features are studied in this section in detail. The workflow scheduling model presented in [14] allows scheduling based on dynamic task priority. Using min-max normalization the intimal dynamic threshold is obtained and based on threshold virtual machines

are mapped with tasks. Though the presented model is simple, compared to the solutions of optimization-based workflow scheduling the performance lags in make span and resource utilization. Gravitational search algorithm presented in [15] considers schedule length and cost ratio for the optimization problem and obtains solution using the gravitational search algorithm by minimizing the cost and schedule length better than HEFT and genetic algorithms.

The improved Firefly Optimization-based workflow scheduling presented in [16] incorporates genetic algorithm operators and quasi reflection learning procedures to enhance the scheduling performances. The objective was considered to reduce the make span and cost and it was attained better than traditional firefly and other optimization algorithms. The Self Adaptive Fruit Fly Optimization (SAFFO)-based workflow scheduling presented in [17] considered cost and make span for the objective function and obtained global solution through the optimization model. The experimentation results report that the presented model effectively reduces the cost and make span over traditional GA, ABC and FFO algorithms.

The optimized workflow scheduling model presented in [18] utilizes Ant Colony Optimization to reduce the cost and make span. The presented model initially utilizes heterogeneous earliest end time for initial scheduling and then utilizes optimization algorithm for optimal scheduling of workflow tasks to physical machines. Experimental results show that ACO based scheduling is better than conventional scheduling procedures. The two-tier workflow model presented in [19] includes linear optimization and gradient decent search-based resource estimation and cluster-based resource scheduling for heterogeneous cloud computing. The objective function considers cost and execution time metrics. The presented optimized scheduling and resource estimation effectively reduces the workflow complexity in comparison to PSO.

The swarm intelligence PSO based workflow scheduling model presented in [20] considers cost, energy consumption and makespan in its objective function. The optimization model converged with non-dominated solutions which provide better exploitation and exploration. Simulation results present the better convergence and performances of optimization model with reduced energy consumption and cost. Considering the cost and make span in their objective function an immune PSO based workflow scheduling is presented in [21]. The presented model overcomes the traditional PSO slow convergence and provides enhanced speed and quality for the scheduling process.

The hybrid optimization model presented in [22] combines PSO with Grey Wolf Optimization (GWO) algorithm to attain better performance in workflow

scheduling. Considering the cost and execution time the optimization model objective function is framed and solved. The hybrid optimization model effectively avoids local optimal trap and provides better global solutions to increase the performance of scheduling process over convention PSO and GWO models. Similar PSO is combined with ant lion optimization (ALO) algorithm as a hybrid model in [23] considering the cost, make span and load. The results present that the hybrid PSO model outperforms in load, make span and cost compared to traditional ALO, PSO and GA-PSO models.

The hybrid ALO based workflow scheduling presented in [24] combines ALO with grasshopper optimization algorithm. The objective function of presented model considered energy consumption, task cost, throughput and make span and obtained optimal solution using hybrid optimization algorithm. The search performance of the optimization algorithm is enhanced using greedy and chaos theory and attained better scheduling performance over conventional optimization algorithms.

The genetic algorithm-based workflow scheduling model presented in [25] utilizes modified genetic operators and load balancing routing to improve the scheduling performances. The presented model utilizes the solutions of heuristic for initial chromosome population and generates the rest of chromosomes based on load balancing routing requirements. The fitness function is considered the cost and make span and effectively reduced using the obtained solution. Similar GA based workflow scheduling presented in [26] considers cost and time for the scheduling fitness function. Instead of considering the heuristics for initial population chromosome, HEFT is used which improves the efficiency of scheduling process over conventional techniques.

Heuristic optimization algorithms-based workflow scheduling is presented as a comparative analysis in [27]. The presented analysis considered cost and make span in the objective function and employed invasive weed optimization (IWO), GA, PSO, and hybrid heuristic optimization algorithms. Experimentation results reveal that the hybrid heuristic optimization algorithm effectively reduces the cost and makespan over traditional optimization algorithms. The workflow scheduling presented in [28] incorporates hybrid chemical reaction optimization (HCRO) algorithm and energy efficient workflow scheduling.

A Hybrid Weighted Ant Colony Optimization Algorithm (HWACOA) is proposed in [29] for task scheduling in cloud computing. For instance, HWACOA achieves a makespan of 13.78 microseconds with 100 tasks and 50.4 microseconds with 400 tasks, compared to ACOs 18.8 microseconds and 66 microseconds, respectively. Similarly, the cost assessment indicates lower task execution costs for HWACOA across various scenarios. The merits

of HWACOA include higher efficiency and faster convergence compared to other algorithms. However, a limitation of the approach is its complexity, which may lead to increased computational overhead under certain conditions, especially as the number of tasks scales up.

The MTD-DHJS (Makespan-Optimized Task Scheduling Algorithm) is introduced in [30] for Cloud Computing with Dynamic Computational Time Prediction) technique, which integrates Dynamic Heuristic Johnson Sequencing (DHJS) and advanced queuing models for task scheduling in cloud environments. The DHJS algorithm decreases the average waiting time by up to 30% compared to other scheduling algorithms. Merits of this approach include increased efficiency in resource allocation, scalability for complex workloads, and reduced processing delays. However, limitations include potential complexity in real-time implementation and the requirement for extensive computational resources in scenarios with rapidly fluctuating workloads.

A hybrid workflow scheduling algorithm combining Particle Swarm Optimization (PSO) and Simulated Annealing (SA) is proposed in [31] for energy-efficient and deadline-aware task scheduling in fog and cloud environments. The technique optimizes task prioritization using PSO and improves task allocation using SA. Key evaluation metrics include energy consumption and makespan. The results show that the PSO-SA algorithm enhances energy efficiency by 5% and reduces makespan by 9% on average compared to the IKH-EFT baseline. Other metrics, such as the percentage of IoT tasks meeting deadlines, were improved by around 99%. The primary merits of the work include its ability to efficiently manage resources, reduce energy consumption, and improve task scheduling to meet deadlines and priorities in a fog-cloud environment. However, the algorithm's complexity could be a limitation, especially when scaling up in large, dynamic fog environments, potentially leading to increased computational overhead.

An Improved Wild Horse Optimization with Levy Flight Algorithm (IWHOLF) is developed in [32] for effective task scheduling in cloud computing environments. This approach integrates the social behaviors of wild horses with the Levy flight theory to improve task scheduling by reducing makespan and maximizing resource utilization. The merits of the IWHOLF technique include improved scheduling efficiency, reduced makespan, and optimal resource utilization. However, the complexity of combining WHO and LF may result in increased computational overhead, particularly in large-scale task environments.

A hybrid multi-objective optimization algorithm (HGSOA-GOA) is presented in [33] for scientific workflow scheduling in multi-cloud environments. The coverage ratio of HGSOA-GOA was also superior, showing

greater efficiency in finding optimal solutions. The merits of this approach include higher accuracy, better distribution of solutions, and improved performance on unbalanced workflows such as Cybershake and Montage. However, a limitation of the algorithm is the increased runtime due to its complexity, which may hinder scalability for extremely large workflows.

A hybrid of particle swarm optimization and gray wolf optimization is presented in [34], named the PSO-GWO algorithm, is proposed for workflow scheduling. The proposed algorithm was tested to reduce the total executing cost (TEC) and total execution time (TET) of the dependent tasks in the cloud computing environment. The proposed algorithm takes advantage of both the standard PSO and GWO algorithms and does not stick to the local optimal solution. The experiment results show that the PSO-GWO outperformed compared with the standard PSO and GWO algorithm in TEC and TET.

Recent studies have also introduced improved variants of evolutionary algorithms for workflow scheduling. In literature [35], proposed a PSO-based scheduling strategy that integrates idle time slot-aware rules to enhance cost and deadline satisfaction. Similarly [36], developed an enhanced PSO algorithm that reduces task dimension and improves initialization to lower execution time and cost. While these methods demonstrate notable improvements, direct comparison with our proposed DSSSO model was not feasible due to differences in objectives, datasets, and benchmark settings.

The reviewed workflow scheduling strategies exhibit several strengths and limitations. Single metaheuristic algorithms such as PSO, ACO, and GA are computationally efficient and relatively simple to implement. They are effective for basic task-to-resource mapping and can optimize individual objectives like cost or makespan. However, they often suffer from premature convergence,

lack robustness in high-dimensional search spaces, and typically ignore critical QoS parameters such as energy consumption and resource utilization. Hybrid models, including PSO-GWO, ALO-GA, and HCRO, improve global search and convergence rates by integrating complementary algorithms. These methods generally perform better across multiple objectives and exhibit improved accuracy and load balancing. Yet, they tend to be computationally complex, parameter-sensitive, and often lack adaptability to dynamic or real-time cloud environments. Additionally, many models are tailored for specific workloads or simulation settings, which raises concerns about scalability and generalizability in real-world deployment. These limitations collectively motivate the need for a more balanced, adaptive, and comprehensive approach like the proposed DSSSO model.

The presented model aimed to minimize the make span and increase the energy efficiency through the optimal solution which is better than traditional GA and HCRO algorithms. The summary of research works is presented in Table 1.

While the above works demonstrate promising advancements in workflow scheduling using various heuristic and metaheuristic models, they often share common limitations. Most notably, a significant number of models, such as those based on PSO, GA, and ACO, optimize for only one or two objectives—primarily makespan and cost—while overlooking other essential QoS parameters like energy consumption and resource utilization. Further, many of the hybrid models, although improving upon local optima avoidance, still exhibit issues of increased complexity, poor scalability in dynamic workloads, or reliance on fixed parameter tuning. For instance, methods like HWACOA and HCRO, while effective in convergence, encounter computational overhead during high task-load conditions. Similarly, models such as

Table 1 Summary of research works

References	Methods	Objective function metrics
[12]	Min-max normalization	Make span and Resource utilization
[13]	Gravitational search algorithm	Cost ratio and Schedule length
[14]	Improved Firefly Optimization	Make span and cost
[15]	Self-Adaptive Fruit Fly Optimization	Cost and make span
[16]	Ant Colony Optimization	Cost and make span
[17]	Linear optimization with gradient decent	Execution time and cost
[18]	Particle Swarm Optimization (PSO)	Cost, Energy consumption and Make span
[19]	Immune PSO	Cost and make span
[20]	PSO with GWO	Cost and execution time
[21]	PSO with ALO	Cost, make span and load
[22]	ALO with grasshopper optimization	Energy consumption, cost, throughput and make span
[23]	Modified Genetic algorithm	Cost and make span
[24]	Genetic Algorithm	Cost and time
[25]	PSO, GA, invasive weed optimization (IWO) and Hybrid heuristic optimization	Cost and Make span
[26]	Hybrid chemical reaction optimization	Make span and Energy efficiency

PSO-GWO and IWHOLF offer strong results in specific metrics but lack adaptability when extended to multi-cloud or large-scale environments. These shortcomings highlight the need for a unified, multi-objective model that ensures exploration-exploitation balance, minimizes overhead, and adapts efficiently to heterogeneous task environments. The proposed DSSSO model is introduced precisely to bridge this gap, offering enhanced convergence stability and broader metric optimization.

From the summary it can be observed that most of the optimization models considered cost and make span in their objective function. However, other factors that affect the QoS of users in workflow scheduling such as energy consumption and resource utilization need to be considered. Thus, a hybrid optimization model is presented in this research work for workflow scheduling considering energy consumption, resource utilization, cost, and make span.

While single metaheuristic algorithms such as PSO, ACO, and GA are computationally efficient, they largely focus on minimizing makespan or cost and neglect other equally important QoS factors like energy efficiency and resource utilization. Hybrid models such as PSO-GWO, ALO-GA, and HCRO offer better convergence and global search ability but often introduce significant computational overhead, parameter sensitivity, and poor adaptability to real-time workloads. Thus, despite numerous advancements, there remains no comprehensive framework that balances exploration and exploitation while optimizing all major QoS metrics together. This shortfall in existing literature forms the central research gap that motivates the design of the proposed DSSSO model, which explicitly integrates multiple objectives with a dual-phase optimization strategy.

While numerous optimization-based workflow scheduling models have been proposed, most of them primarily focus on minimizing makespan and cost, often neglecting equally critical parameters such as energy consumption and resource utilization. For instance, approaches based on Particle Swarm Optimization (PSO), Genetic Algorithms (GA), and Ant Colony Optimization (ACO) demonstrate efficiency in reducing execution time and cost but fail to comprehensively address energy-aware scheduling requirements. Similarly, hybrid models such as PSO-GWO, ALO-GA, and HCRO improve global search and convergence but often suffer from high computational overhead, parameter sensitivity, and poor adaptability in dynamic workload conditions. Furthermore, many existing studies are tailored to specific workloads or simulation settings, which limits their scalability and generalizability to large-scale heterogeneous cloud environments [37, 38].

These limitations collectively highlight a clear research gap: the absence of a unified, multi-objective workflow

scheduling framework that simultaneously optimizes makespan, cost, energy consumption, and resource utilization while maintaining a robust balance between exploration and exploitation. To bridge this gap, the present research introduces the Dolphin Swarm Sparrow Search Optimization (DSSSO) algorithm, which integrates the global exploration capabilities of Dolphin Swarm Optimization with the local refinement strengths of Sparrow Search Optimization. This hybrid structure ensures convergence stability, avoids premature local optima, and delivers superior adaptability to heterogeneous and large-scale environments. By explicitly incorporating energy and resource-aware objectives along with traditional performance metrics, the DSSSO framework provides a more comprehensive, scalable, and practically viable solution compared to existing models

Proposed work

Cloud workflow scheduling environment

Workflows are used to model scientific applications and it requires high performance computing environment. Generally, workflows are modeled as set of interconnected tasks with more data or computing dependencies. A cloud workflow management system in cloud executes this workflow application. This management system has the responsibility to manage, schedule and execute the workflows based on SLA and user-specified QoS requirements. In the workflow engine, the data management component performs data storage and movement as per requirement. While executing workflow resource provisioning and scheduling are considered important stages. The resources required to execute the task are selected and provided in the resource provisioning stage and while scheduling the tasks are mapped into the best available resources.

The proposed high-level architecture given in Fig. 1 has the workflow management system and it utilizes cloud resource while executing the scientific workflow application. The elements in the architecture are workflow engine, workflow planner, workflow scheduler and clustering engine. The proposed multi-objective hybrid optimization algorithm is used in the workflow planner so that suitable cloud service providers can be identified to the users and data center loads can be tracked while allocating resources so that the necessary quality of service can be maintained. The proposed hybrid optimization-based workflow scheduling model includes metaheuristic dolphin swarm and sparrow search optimization algorithms. The proposed hybrid Dolphin Swarm Sparrow Search Optimization (DSSSO) was obtained to effectively solve the multi-objective cloud workflow scheduling problem. Dolphin Swarm Optimization (DSO) was selected for its robust exploration abilities. The dolphins' echolocation and cooperative hunting behaviors of DSO

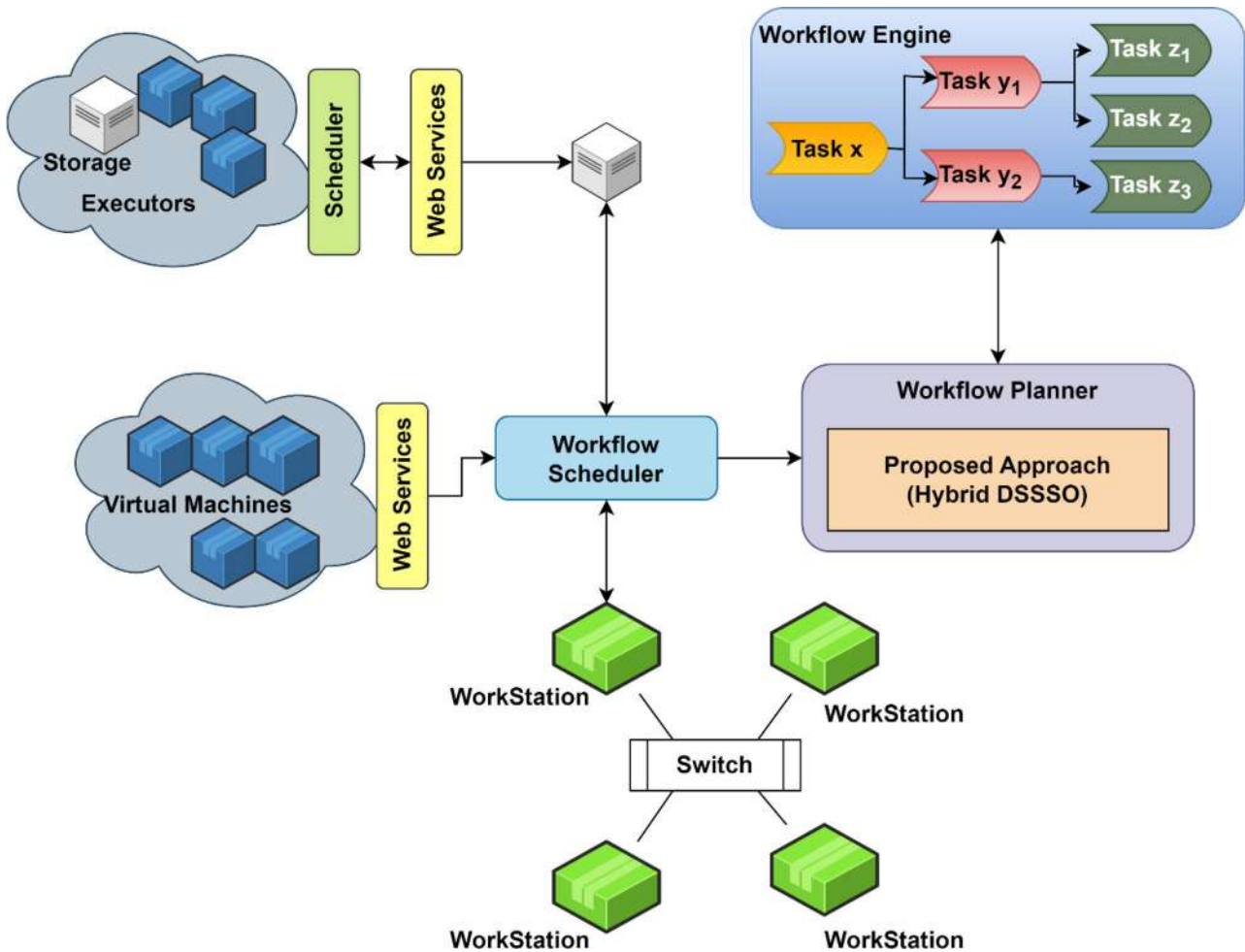


Fig. 1 High level architecture for workflow scheduling

to avoid local optima solution in search space. Whereas the Sparrow Search Optimization (SSO) algorithm was incorporated for its strong exploitation features which formulates the foraging and predator-avoidance behaviors of sparrows. This allows the algorithm to fine-tune solutions efficiently and improve convergence rates. The hybridization of DSO and SSO utilizes the strengths of both optimization model. DSOs exploration ensures the model does not prematurely converge, while SSAs exploitation accelerates convergence by refining the best solutions. This combination makes DSSSO highly effective in addressing the complexity and scalability issues in workflow scheduling for cloud environments outperforming traditional optimization approaches.

Cloud service providers have data centers in different geographical locations and provide their services to users through the internet without violating the service level agreement. The data centers will have virtual machines and it is considered as a computing resource. Each virtual machine has different characteristics and different computing features. Memory capacity, network bandwidth,

processing cores and cycle times will be different for each virtual machine. The SLA specified either by user or service provider provides the details of service and QoS parameters. Violating the expectations will cause penalty for both parties. Multi-task applications should run smoothly on virtual machines and it is defined based on the number of cores required to process the user tasks. In the proposed model mathematical formulation, a set of interdependent tasks of workflow application are considered. The tasks need to be executed with correct sequential order on a heterogeneous virtual machines. By using direct acyclic graph, the workflow application is modeled. Consider if there are N virtual machines then it is represented as $N_{pm} = n_{pm1}, n_{pm2}, n_{pm3}, \dots, n_{pmn}$. The tasks are represented as tuple $t_j = \{m, j, p_e\}$ which provides the details like identifier (j), length of identifier (m) in million instruction and number of processors (p_e) running for task on virtual machine.

In the application model, workflows are represented using direct acyclic graph in which the graph vertices represent the tasks and graph edges represents the

relation between the tasks. The task computation load is defined in million instructions and the amount of output data generated by previous task is represented through a label on each edge. The entry task doesn't have any previous task labels and the final task doesn't have a descendant. The proposed model considered a workflow with 15 tasks in which each task must wait for the previous task output before performing their own.

Mathematical formulation of objective function

The objective of the proposed hybrid optimized scheduling model is to improve resource utilization and minimize energy consumption, make span and cost. This must be achieved by properly assigning each workflow tasks with its complex task relationships to physical machines. Before assigning the necessary features like task precedence, availability of resources and size of the workflow are assumed initially and established as an NP-hard problem. The workflow scheduling problem is modeled as a multi-objective optimization task where the goal is to minimize makespan, cost, and energy consumption while maximizing resource utilization. The general objective function is formulated as follows:

$$\min h = \sum_{v=1}^x \sum_{i=1}^y \sum_{j=1}^z t_{pv} t_{a_{ij}} o_v \quad (1)$$

subject to

$$\sum_{j=1}^z t_{a_{ij}} = 1, i = 1, 2, \dots, y \quad (2)$$

$$\sum_{v=1}^x t_{pv} = 1, 0 \leq t_{pv} \leq 1 \quad (3)$$

Where $i = 1, 2, 3 \dots y$: index of tasks in the workflow.

$j = 1, 2, 3 \dots z$: index of virtual machines (VMs)

$v = 1, 2, 3 \dots x$: index of physical machines (hosts)

Decision variables

$t_{a_{ij}} \in \{0, 1\}$: task allocation variable, equals 1 if task i is assigned to VM j , and 0 otherwise.

- $o_v \in \{0, 1\}$: host activation variable, equals 1 if physical machine v is active (i.e., at least one task is assigned to a VM running on it), and 0 otherwise.
- $s_i > 0$: start time of task i .
- $f_i \geq 0$: finish time of task i .

Parameters

- t_{pv} : unit cost or power consumption rate of physical machine v .
- $et_{i,j}$: execution time of task i on VM j .
- $bw_{j,l}$: communication bandwidth between VM j and VM l .
- $out_{p \rightarrow i}$: data size transferred from predecessor task p to task i .

If the physical machine is allocated with a workflow task, then $t_{a_{ij}}$ is considered as 1 otherwise it is considered as 0 based on the conditions given in Eqs (2) and (3). The sum of t_{pv} is limited to 1 and it is used as a balancing factor between resource utilization, cost, energy consumption and make span.

Dolphin swarm optimization phase

The proposed workflow scheduling model starts with user task provided by the cloud users and the task are scheduled using the proposed hybrid Dolphin swarm sparrow search optimization (DSSSO) algorithm. The dolphin swarm optimization is framed based on the lifestyle habits of dolphins. The real predatory system of dolphins is to chase and attack prey in the sea which is formulated as an optimization model. The predation process has three phases in which dolphins begin the hunting process by creating noises and following it up with the echolocation to discern the surrounding. The attacks are performed either by flip swimming mode or rotational swimming mode. If a dolphin identified with a large food source, then it will call their friends for help and finally finish the predation. In simple terms these three phases are given as (i) target location detection, (ii) chasing and approaching the target, (iii) attacking the prey. In the proposed work, the workflows are related to dolphins and the optimal solution is considered as final scheduling results.

In the optimization initialization phase, dolphins are represented as $[d_1, d_2, \dots, d_n]$ and for each dolphin two variables l_i and k_i are assigned to represent the best solution obtained at single instant (l_i) and the best solution (k_i) obtained from others respectively. The initial position of dolphin is defined as

$$x_d = (x_{d_1}, x_{d_1}, \dots, x_{d_n}) \quad (4)$$

where the position vector of d^{th} dolphin in search space is indicated as x_d . The dolphins are randomly initialized with boundary search space of (x_{max}, x_{min}) . In the search space, dolphins explore their surroundings and make noises in arbitrary directions which is described as $U_i = [u_{d_1}, u_{d_2}, \dots, u_{d_n}]$. The total noise in every dimension and noise direction is collectively termed using U_j . When

the dolphin renders a sound U_j at t , it starts to search for new solution which is mathematically given as

$$d_{ijt} = d_i - U_{jt} \quad (5)$$

In the echolocation-based exploration phase, dolphins emit sound waves to explore the environment and adjust their positions accordingly. Each dolphin updates its position based on the current position, the best global solution, and a random noise factor for exploration. The position update process is mathematically formulated as

$$x_d(t+1) = x_d(t) + \alpha \cdot (x_{best}(t) - x_d(t)) + \beta \cdot N(0,1) \quad (6)$$

where the new position of the d^{th} dolphin is indicated as $x_d(t+1)$, best position is indicated as $x_{best}(t)$. α and β indicates the learning factor and random scaling factor respectively. Random noise in normal distribution is indicated as $N(0,1)$. Further the fitness of each dolphin is evaluated considering the objective function of the proposed scheduling problem as follows.

$$fit = tp_1 m_s + tp_2 c_w + tp_3 e_c + tp_4 \frac{1}{Tr_{pu}} \quad (7)$$

where tp is the tuning parameter, m_s indicates the make span, c_w indicates the workflow cost, e_c indicates the energy consumption and r_{pu} indicates the resource utilization. The tuning parameters tp_1 , tp_2 , tp_3 and tp_4 are essential in adjusting the balance between the make span (m_s), workflow cost (c_w), energy consumption (e_c), and resource utilization (r_{pu}). These values are selected after analyzing the effect of varying each parameter on the objective function and not fixed randomly. Since the fitness function of DSO is used to quantify the objective function, the better solution will effectively minimize the objective function parameters.

The second phase has two processes as reception and call process in which reception occurs only after the call process. In the optimization model a transmission time matrix is used to preserve the reception and call process interchange. The transmission time matrix is described using T_{ij} and it exactly defines the time remaining for the noise to pass through dolphin i to dolphin j . It is essential to reduce the transmission time matrix into one till the optimization reaches the reception process. If $T_{ij} = 0$, then d_i hears the noise send from d_j . Upon receiving the noise, the transmission time matrix is replaced with full transmission time and the fitness is evaluated again. In the call process, all the dolphins make noise to alert others thus the transmission time matrix is checked as $T_{ij} > \left\lceil \frac{Dd_{i,j}}{A.Speed} \right\rceil$, where $Dd_{i,j}$ is the distance between

dolphin d_i and d_j , the acceleration constant is given as A and the noise speed attribute is indicated using $speed$.

In the final predation phase, each dolphin should measure the encompassing sweep r_2 . Before that for each dolphin the look span radius r_1 is calculated as

$$r_1 = T_1 \times speed \quad (8)$$

where T_1 indicates the peak search time. Further, the encompassing sweep measurement is presented based on the dolphin position in three cases. For demonstrating the three cases, the position of dolphin, look span radius r_1 , encompassing sweep r_2 and the best solution Dk_i obtained from others are considered. In the first case, $Dk_i \leq r_1 \times speed$ is considered based on that the encompassing span is measured using the expression given below.

$$r_2 = \left(1 - \frac{2}{e}\right) Dk, \text{ for } e > 2 \quad (9)$$

where the radius diminishing coefficient is given as e . Based on the obtained encompass sweep, the dolphin's new position is obtained as

$$nd_i = k_i + ((d_i - k_i) / Dk_i) * r_2 \quad (10)$$

where nd_i indicates the dolphin's new position. In the next cases, the $Dk_i > r_1$ and $Dk > Dkl_i$, the dolphin accepts the information from other dolphin and the encompassing sweep is calculated as

$$r_2 = \left(1 - \frac{(Dk_i / fit) - (Dk_i - Dkl_i / fit)}{e Dk (1 / fit)}\right) Dk_i, \text{ for } e > 2 \quad (11)$$

where Dkl_i indicates the distance between the solution obtained at single instant l_i and best solution k_i obtained from others. Based on this, the new position is updated as

$$nd_i = k_i + (rand || rand) r_2 \quad (12)$$

In the last case, the dolphin moves to an arbitrary position and its condition is similar to $Dk_i > Dkl_i$ and the encompassing sweep and new position are obtained based on Eq. (9) and (10). Figure 2(a)-(c) depicts the initial condition of all the three cases and Fig. 3(a)-(c) depicts the position updated results of all the cases.

Based on these three cases, the dolphin encompassing sweep and new positions are obtained. Once the dolphin moved into a new position, the position wellness is compared with best solution (k_i) and updated if the new position is better than the best solution. Otherwise, the current position is considered best, and the process is

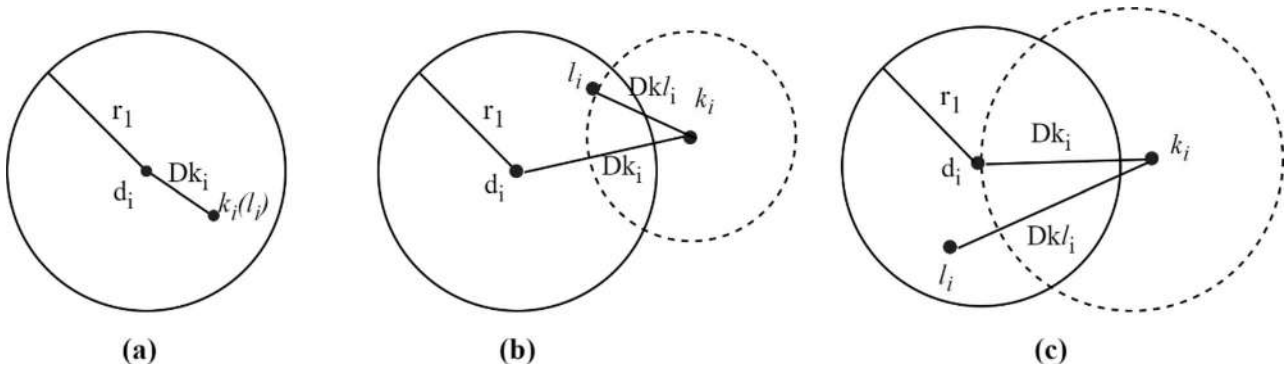


Fig. 2 (a)-(c) Initial condition of all the three cases

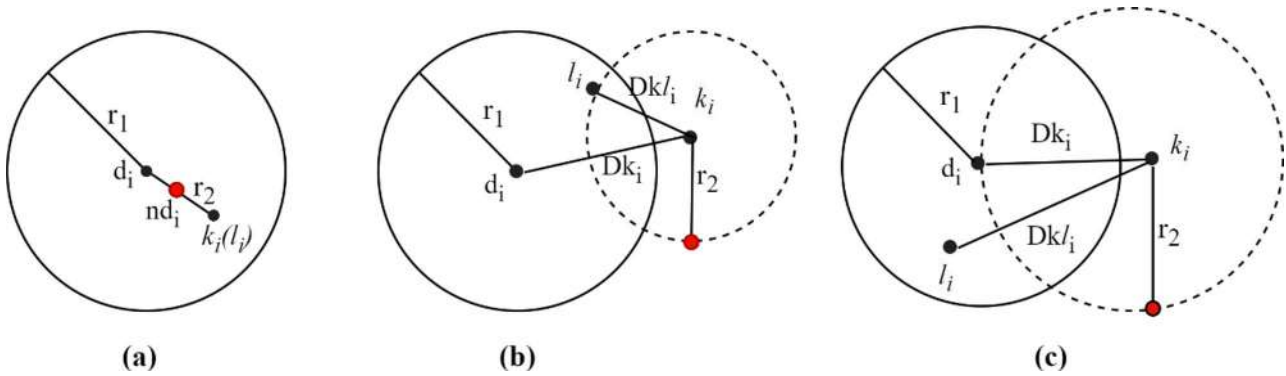


Fig. 3 (a)-(c) Updated results for all the three cases

repeated till it meets the termination criteria. To improve the performance of dolphin swarm optimization and avoid premature convergence, sparrow search optimization algorithm is incorporated in the proposed work. The process flow of proposed hybrid DSSSO optimization is presented as follows in Fig. 4.

Sparrow search optimization phase

The food foraging and predator avoidance features of sparrows are used to frame the optimization algorithm. The optimization algorithm considered two types of sparrows such as scroungers and makers, in which the makers will have high energy levels and select the food foraging location or provide instructions to all scroungers. While adopting the foraging strategies, the energy levels play an important role. The birds positioned in the search boundaries are more likely to be targeted by the predators. Birds located in the middle move close to their neighbors to minimize the predator attack. The sparrows have an instinct for the danger so when one bird identifies the danger then all birds will take flight and move into another location. Mathematically the sparrow habitat is expressed as a matrix as follows.

$$\begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,d} \\ x_{2,1} & x_{2,2} & \dots & x_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n,1} & x_{n,2} & \dots & x_{n,d} \end{bmatrix} \quad (13)$$

where the number of sparrows is indicated using n and the fitness of each sparrow is obtained as

$$\begin{bmatrix} f[x_{1,1} & x_{1,2} & \dots & x_{1,d}] \\ f[x_{2,1} & x_{2,2} & \dots & x_{2,d}] \\ \vdots & \vdots & \ddots & \vdots \\ f[x_{n,1} & x_{n,2} & \dots & x_{n,d}] \end{bmatrix} \quad (14)$$

Based on the current position and fitness value, the food location is updated for every cycle as follows.

$$x_{i,j}^{t+1} = \begin{cases} x_{i,j}^t e^{(-i/\alpha iter_{max})} & \text{if } r < s_r \\ x_{i,j}^t + \varphi \vartheta & \text{if } r \geq s_r \end{cases} \quad (15)$$

where α is a random value and its range is given as $[0,1]$, s_r indicates the safety threshold and its range is given as $[0.5, 1]$ and the alarm value is indicated using r and its range is given as $[0,1]$. φ is a random number which follows normal distribution and ϑ is used to indicate the matrix with dimension of $1 \times d$ with each element inside the matrix is 1. When $r < s_r$, then there are no predators

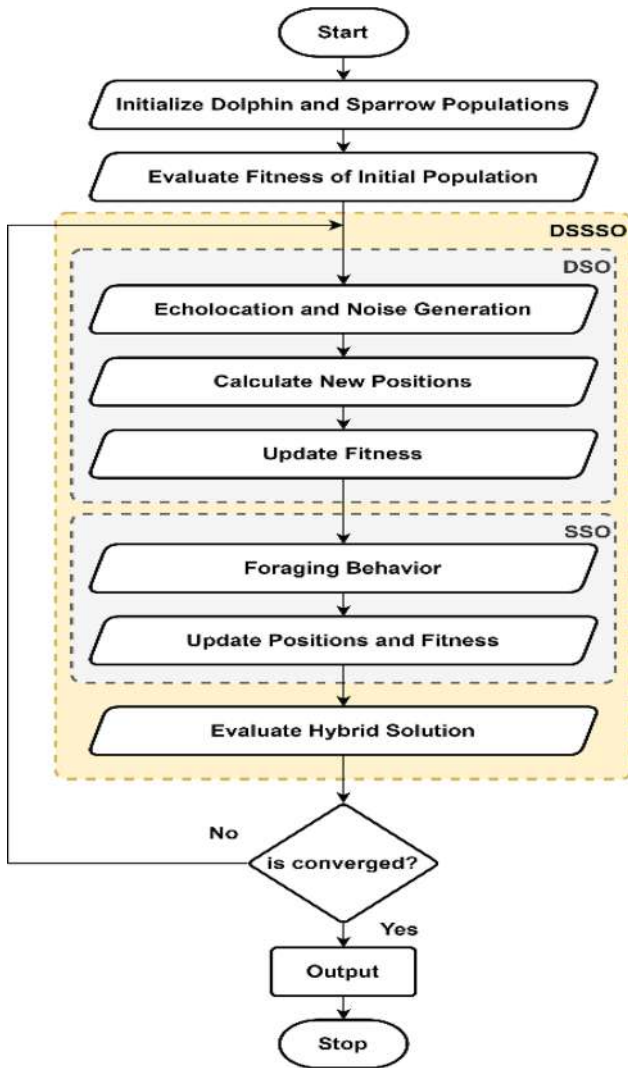


Fig. 4 Flowchart of proposed hybrid DSSO

around the region so that the birds will encounter a wide search for food and if $r \geq s_r$, then some sparrows identified predators and all the sparrow need to fly from the current place to safety place immediately. In the food searching process, some scroungers monitor the makers frequently. If the makers could not be able to identify enough food, then scroungers will fly to another location and search for the food. If the scroungers identified food in the new location, they get the food to makers immediately otherwise the process is continued. The position update of scroungers is mathematically formulated as

$$x_{i,j}^{t+1} = \begin{cases} \varphi e^{\left(\frac{x_{worst}^t - x_{i,j}^t}{i^2}\right)} & \text{if } i > n/2 \\ x_p^{t+1} + |x_{i,j}^t - x_{i,j}^{t+1}| A^+ \vartheta & \text{otherwise} \end{cases} \quad (16)$$

where the position occupied by the predator is indicated using x_p and the global worst location is indicated using

x_{worst} . A represents a matrix in which each element is randomly assigned with +1 or -1. If the sparrows at the edge are aware of danger and move towards a safe location while the middle sparrow randomly walk close to others. This condition is mathematically framed as

$$x_{i,j}^{t+1} = \begin{cases} x_{best}^t + \beta |x_{i,j}^t - x_{best}^t| & \text{if } f_i > f_g \\ x_{i,j}^t + g \left(\frac{|x_{i,j}^t - x_{worst}^t|}{(f_i - f_w) + \rho} \right) & \text{if } f_i = f_g \end{cases} \quad (17)$$

where the current global optimal location is indicated using x_{best}^t , the step size control parameter is given as β , g is the random number and its range is given as [0,1]. The current sparrow fitness is indicated using f_i and its global best and worst fitness values are indicated using f_g and f_w respectively. A small constant ρ is added to avoid zero division error. The condition $f_i > f_g$ defines that the sparrow is at the edge and the location of center population which is safe is indicated through the x_{best}^t . The condition $f_i = f_g$ describes that the sparrows in the middle are aware of danger and move closer to others and g describes the direction of movement and the step size.

Hybrid DSSSO flow and final task assignment

Based on the optimal solutions the conditions the new positions of dolphins are adjusted to obtain the optimal performance in scheduling application.

Brief workflow is depicted in Fig. 5.

In the proposed DSSSO-based workflow scheduling model, task-to-node assignment is handled through a two-phase optimization process involving Dolphin Swarm Optimization (DSO) for global exploration and Sparrow Search Algorithm (SSA) for local refinement. Initially, the entire workflow is modeled as a Directed Acyclic Graph (DAG), where each node represents a task and edges denote data dependencies. The cloud environment is represented as a set of heterogeneous virtual machines (VMs), each with specific computational capabilities. The DSO phase generates a population of potential scheduling solutions, where each individual represents a complete mapping of tasks to VMs. During this phase, the fitness of each solution is evaluated based on a multi-objective function considering makespan, energy consumption, cost, and resource utilization. Solutions are refined through echolocation-based position updates and cooperative swarm behaviors. The best-performing solutions are passed to the SSA phase, which focuses on enhancing local exploitation. Here, task assignments are further fine-tuned based on dynamic safety thresholds and predator-avoidance behaviors, ensuring high-quality VM allocations. Tasks are assigned to VMs not only based on availability and execution time but also considering QoS constraints like minimal idle time, optimal power usage, and resource saturation

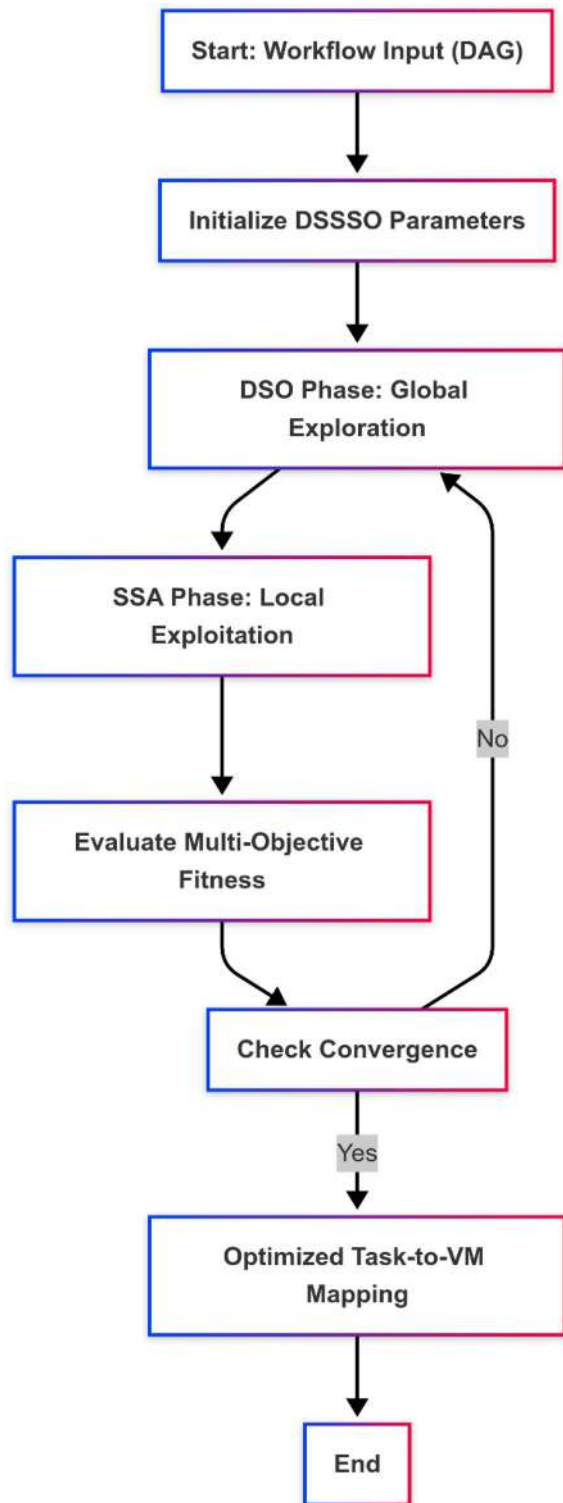


Fig. 5 Condensed process flow of the proposed DSSSO algorithm, integrating global exploration (DSO phase) and local exploitation (SSA phase) for optimized multi-objective workflow scheduling

levels. This results in a highly optimized and adaptive task-to-node assignment strategy that balances performance, efficiency, and scalability in real-time cloud environments.

The summarized pseudocode for the proposed hybrid optimization is presented as follows.

Pseudocode for the proposed hybrid optimization algorithm

Initialize the dolphin swarm in d -dimensional space and initialize the sparrows
Begin

Calculate the fitness function for each dolphin using Equation (4)

Initialize call phase

$$T_{ij} = \begin{cases} \frac{Dd_{i,j}}{A.Speed} & \text{if } fit_{d_i} < fit_{d_j} \text{ and } T_{ij} > \left\lceil \frac{Dd_{i,j}}{A.Speed} \right\rceil \\ T_{ij} & \text{otherwise} \end{cases}$$

Initialize reception phase

T_{ij} decreased to one

$$fit_{d_i} = \begin{cases} fit_{d_i} & \text{if } fit_{d_i} < fit_{d_j} \text{ and } T_{ij} = 0 \\ T_{ij} & \text{otherwise} \end{cases}$$

Initialize predation phase

Obtain the distance Dk_i and Dkl_i

If $Dk_i \leq r_1$

$$r_2 = \left(1 - \frac{2}{e}\right) Dk$$

else

$$Dk_i > Dkl_i, \quad x_{i,j}^{t+1} = \{x_{best}^t + \beta[x_{i,j}^t - x_{best}^t]\}$$

else

$$x_{i,j}^{t+1} = x_{i,j}^t + g \left(\frac{|x_{i,j}^t - x_{worst}^t|}{(f_i - f_w) + \rho} \right)$$

End if

Update the position of dolphin and calculate the fitness

End while

Select the best solution

End

End

In the proposed hybrid Dolphin Swarm Sparrow Search Optimization (DSSSO) algorithm the convergence occurs in distinct phases. Initially, the global convergence begins during the DSO phase, where dolphins explore a wide range of solutions across the solution space. This exploration ensures that the algorithm does not prematurely focus on local optima and explores diverse regions of the solution space. The convergence toward global optima occurs when the dolphins, through echolocation and cooperative hunting strategies begin to cluster around the most promising areas of the solution space. This happens as the DSO phase progresses, and more dolphins move toward higher-quality solutions based on the multi-objective fitness function. The global convergence primarily takes place in the early to mid-stages of the optimization process when the algorithm has sufficient diversity in the solution pool to explore the global search space effectively.

The local convergence follows in the SSA phase, which builds on the globally optimal regions identified by the DSO. During this phase, sparrows refine the solutions further by focusing on exploitation. They adjust their positions based on the best solutions found by the DSO

and use foraging and predator-avoidance behaviors to fine-tune these solutions. The convergence toward local optima occurs when the sparrows gather around the most refined solutions within the smaller regions of the search space. This happens later in the optimization process as the SSA works to improve the precision of the solutions found during global exploration. Thus, the proposed hybrid DSSSO algorithm converges globally during the DSO phase and identifies high-quality areas of the solution space. Whereas the proposed hybrid DSSSO locally converges during the SSA phase in which the solutions are fine-tuned within these areas. This ensures that the algorithm provides a balance between exploring diverse possibilities and exploiting the best-found solutions for optimal results.

Table 2 Simulation hyperparameters

S. No	Algorithm	Parameter	Value/Range
1	Improved Bat Optimization (IBOA)	Loudness (A)	0.5
2		Pulse Rate (r)	0.8
3		Population Size	50
4		Frequency Range	1.5
5	Hybrid Bat Optimization (HBOA)	Initial Population	50
6		Pulse Rate	0.7
7		Loudness	0.4
8		Iterations	100
9	Dolphin Swarm Optimization (DSO)	Population Size	50
10		Echolocation Frequency	0.8
11		Search Radius	15
12		Maximum Iterations	100
13	Sparrow Search Algorithm (SSA)	Initial Population	50
14		Safety Threshold (st)	0.6
15		Explorers-to-Scroungers Ratio	70:30
16		Iteration Count	100
17	Particle Swarm Optimization (PSO)	Population Size	50
18		Cognitive Coefficient (c1)	1.5
19		Social Coefficient (c2)	1.5
20		Inertia Weight (w)	0.7
21	Ant Colony Optimization (ACO)	Maximum Iterations	100
22		Population Size	50
23		Evaporation Rate (p)	0.5
24		Pheromone Influence (α)	1.0
25	Whale Optimization Algorithm (WOA)	Heuristic Influence (β)	2.0
26		Maximum Iterations	100
27		Population Size	50
28		Convergence Control Parameter (a)	1.5
29	Hybrid Dolphin-Sparrow Optimization (DSSSO)	Maximum Iterations	100
30		Encircling Coefficient (b)	1.2
31		Population Size	50
32		Convergence Control Parameter (β)	0.4
33		Maximum Noise Distance	20
34		Search Space Reduction Rate	Dynamic

Results and discussion

The performance analysis of the proposed optimized workflow scheduling model is presented through simulation analysis. Using Workflow Sim tool the scheduling process is simulated. Additionally using workflow generator toolkit, the workflow is generated and scheduled using the proposed optimization algorithm solutions. The number of tasks considered for analysis is 2000 and the metrics considered for performance evaluation are make span, cost, energy consumption and resource utilization. To validate the proposed model performance through comparative analysis, existing bat optimization algorithm, improved bat optimization, dolphin swarm optimization, sparrow search optimization are considered. The results for Bat Optimization Algorithm (BOA) and Improved Bat Optimization Algorithm (IBOA) are obtained from Sobhanayak.et.al., 2023 [39] research work. The conventional PSOA, ACOA, WOA, DSO and SSO are experimented on their own, and their performance is noted to provide the final comparative analysis. Table 2 provides the details of simulation hyperparameters for all the optimization algorithms.

The simulation parameters listed in Table 2 are the final optimal values which provide better performance while applying for the workflow scheduling experimentation. Specifically, the population size is set to 50 to ensure a diverse solution space while balancing computational complexity. The population size of 50 provides sufficient diversity without overloading the system. Similarly, the maximum iteration is set as 100 to ensure the optimization models converge to optimal solutions without consuming too much computational time. The pulse rate in IBOA and HBOA are selected as 0.8 and 0.7 to provide balance between exploration and exploitation. Also, pulse rate ensures that bats perform frequent updates and makes the algorithm to converge faster by shifting towards local exploitation as the number of iterations increases. The loudness in IBOA and HBOA are selected as 0.5 and 0.4 reduces noise during search process. In the case of DSO and SSA the population is set as 50 to ensure diversity. The echo frequency of DSO is selected as 0.8 to enhance the algorithm adaptability. While the safety threshold of SSA is selected as 0.6 to avoid predation. For the proposed DSSSO the control parameter is selected as 0.4 to adjust the balance between exploitation and exploration. The search space reduction rate is considered dynamic. The search space shrinks over time as the algorithm converges.

In order to obtain the optimal values for the optimization algorithms a series of experiments were conducted and the key parameters were adjusted. The major parameters which are adjusted are loudness and pulse rate for IBOA and HBOA. The echolocation frequency and search radius of DSO is varied between 0.5 and 1.0. The

chosen value of 0.8 was found to be ideal for balancing global search capabilities with local exploitation. The convergence control parameter of hybrid DSSSO was tested within the range of 0.1 to 1.0. Dynamic adjustment between 0.3 and 0.9 provided the best results, allowing for a smooth transition from exploration to exploitation.

The proposed workflow scheduling model performance is evaluated through metrics like make span, computation cost, resource utilization and energy consumption. The make span is calculated based on the finish time of the exit node which is formulated as

$$Makespan(m_k) = E.Node_{ft} \quad (16)$$

where exist node is indicated using $E.Node$ and ft indicates the finish time. The cost metric is used to analyze the expenses for utilizing the resources of cloud service providers. The charges are calculated on a second basis and the task completion time is the final time taken by the application. The cost to perform a task in a physical machine is calculated considering the execution time multiplied with physical machine charge rate per second. Mathematically the cost is expressed as

$$c_w = \sum_{i=1}^n (Task_{ft} - Task_{st}) \times Pm_{ch} \quad (17)$$

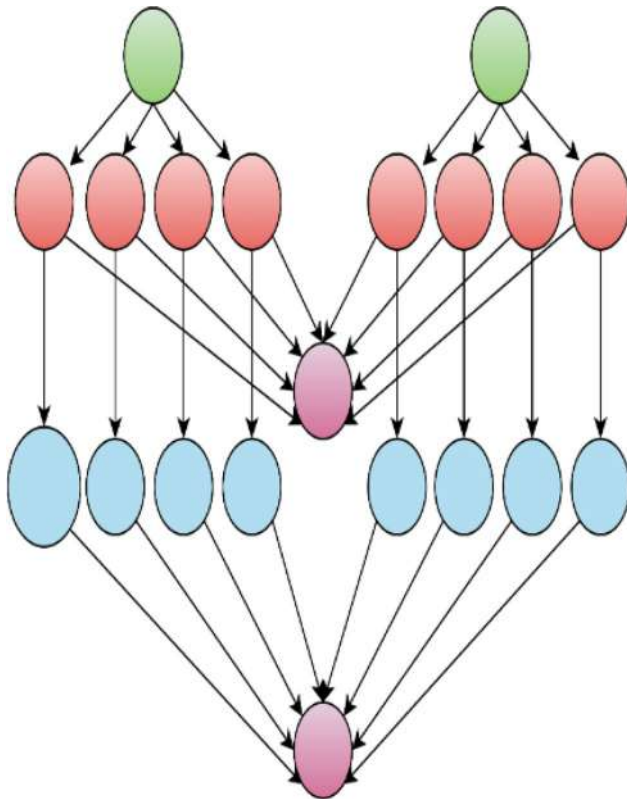


Fig. 6 Cybershake workload

where starting and finishing time of task is indicated using st and ft , the running cost of physical machine is indicated using Pm_{ch} and the total cost is indicated using c_w . The energy consumption is calculated considering the physical machine energy utilization. All the tasks are performed in physical machine processing units and the overall usage is calculated as follows.

$$p_{tot} = p_{min} + (p_{max} - p_{min})\mathcal{U} \quad (18)$$

where total power consumption is represented as p_{tot} , the maximum power utilized during full capacity operations is indicated using p_{max} and the minimum usage is indicated using p_{min} . The processing unit usage is a time dependent quantity which is indicated using $\mathcal{U}$. From this energy consumption is calculated as follows.

$$e_c = \int_{t_0}^{t_1} p(\mathcal{U}_i(t)) \quad (19)$$

The resource utilization is calculated based on the physical machine CPU load in the data center which is formulated as

$$r_{pu} = \sum_{k=1}^n usage(k) / CPU_i \quad (20)$$

where the number of active tasks is indicated using n and the capacity of CPU is indicated using CPU_i . for n physical machines, the CPU load is expressed as

$$Tr_{pu} = \sum_{i=1}^n r_{pu}/n \quad (21)$$

The resource utilization should be maximized to attain better performance in workflow scheduling. The performance testing and validation process of all the models considers Cybershake, Montage, Epigenomics and Ligo workloads. Figures 6 and 7 depict the workloads of cybershake and montage respectively. Figures 8 and 9 depict the Epigenomics and Ligo workloads.

Energy consumption analysis

The energy consumption analysis of proposed model for cybershake and montage workloads are comparatively analyzed with existing scheduling algorithms. Table 3 provides the numerical values of energy consumption details for 2000 tasks through different workloads. The maximum performance is eventually visible from the numerical results compared to existing methods.

Figure 10 depicts the energy consumption of proposed and existing optimization algorithms for workloads

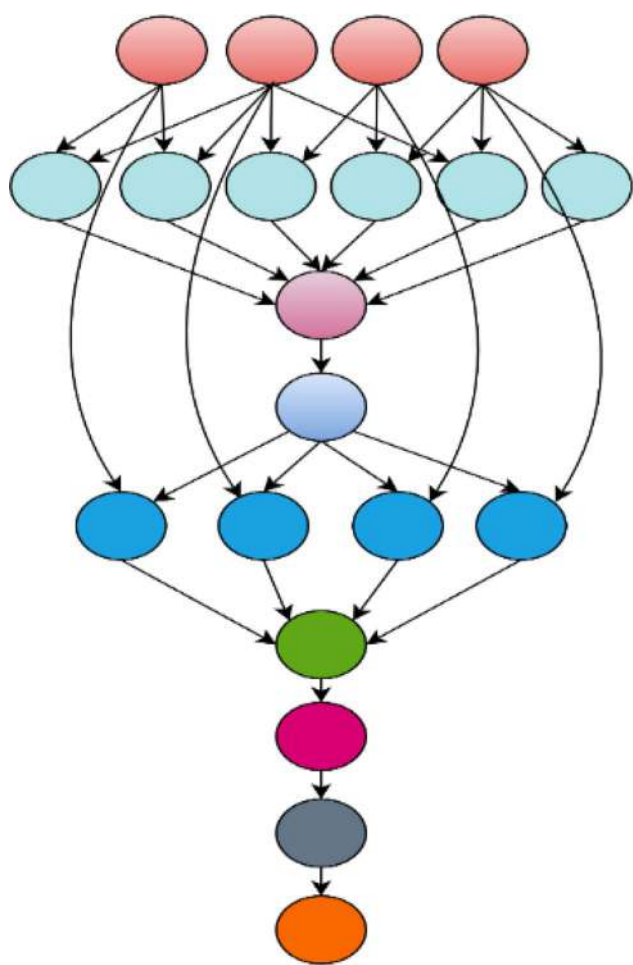


Fig. 7 Montage workload

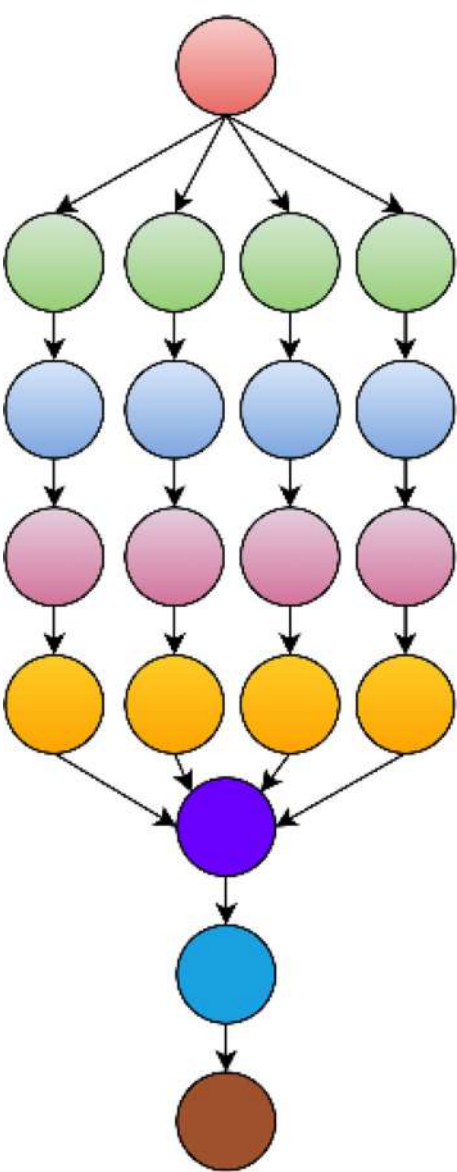


Fig. 8 Epigenomics workload

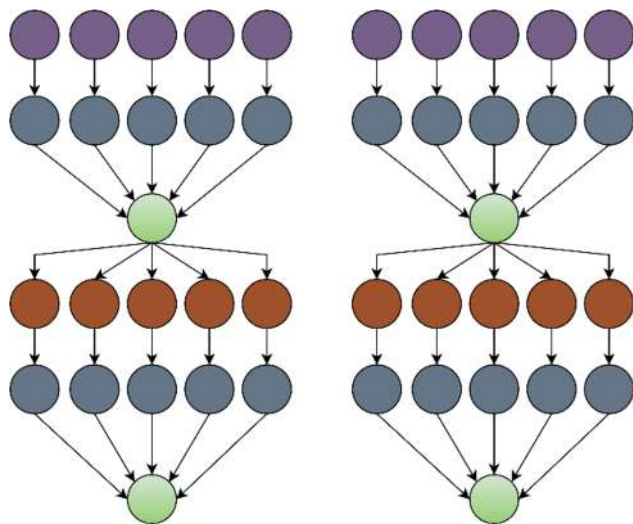


Fig. 9 Ligo workload

like CyberShake, Montage, Epigenomics, and LIGO with an increasing number of tasks. Across all workloads, the proposed HDSSSO consistently exhibits the

Table 3 Energy consumption analysis for workloads

S. No	Methods	Tasks (2000) vs Energy Consumption (J)			
		Cybershake	Montage	Epigenomics	Ligo
1	PSOA	2.9×10^5	4.4×10^5	3.5×10^5	4.2×10^5
2	ACOA	3.1×10^5	4.7×10^5	3.6×10^5	4.3×10^5
3	WOA	2.9×10^5	4.6×10^5	3.4×10^5	4.1×10^5
4	HBOA	3.0×10^5	4.5×10^5	3.4×10^5	4.0×10^5
5	IBOA	5.1×10^5	6.2×10^5	5.4×10^5	6.1×10^5
6	DSOA	2.8×10^5	4.2×10^5	3.3×10^5	3.9×10^5
7	SSOA	2.7×10^5	4.2×10^5	3.2×10^5	3.9×10^5
8	Proposed HDSSSO	2.4×10^5	4.0×10^5	3.1×10^5	3.8×10^5

lowest energy consumption. For the CyberShake workload the proposed HDSSSO consumes 2.4×10^5 J for 2000 tasks, while PSOA, ACOA, WOA, HBOA approximately consume 3.0×10^5 J. The energy consumption of DSOA and SSOA are 2.8×10^5 and 2.7×10^5 respectively. The IBOA exhibits the highest energy consumption of 5.1×10^5 J. Similarly, in the Montage workload the proposed HDSSSO maintains lower energy consumption at 4.0×10^5 J, while PSOA, and HBOA approximately consume 4.5×10^5 J. The energy consumption of ACOA and WOA are approximately 4.7×10^5 J while the energy consumption of DSOA and SSOA are approximately 4.2×10^5 J. The IBOA exhibits the highest energy consumption of 6.2×10^5 J. For the Epigenomics workload the proposed HDSSSO maintains lower energy consumption at 3.1×10^5 J, while PSOA, and ACOA approximately consume 3.5×10^5 J. The energy consumption of HBOA and WOA are 4.5×10^5 J while the energy consumption of

DSOA and SSOA are approximately 3.3×10^5 J. The IBOA exhibit the highest energy consumption of 5.4×10^5 J. For the Ligo workload the proposed HDSSSO maintains lower energy consumption at 3.8×10^5 J, while PSOA, and ACOA approximately consume 4.3×10^5 J. The energy consumption of HBOA and WOA are 4.1×10^5 J while the energy consumption of DSOA and SSOA are approximately 3.9×10^5 J. The IBOA exhibit the highest energy consumption of 6.1×10^5 J. The limitations of the existing models, such as PSO, ACO, and WOA, are apparent in their higher energy consumption, which results from slower convergence and higher computational overhead during task-to-VM assignment. In contrast, IBOA being effective in exploration but suffers from excessive energy consumption due to extensive iterations. The lower energy consumption of proposed HDSSSO across all workloads highlights its efficient task scheduling due to its hybrid nature combining exploration and exploitation

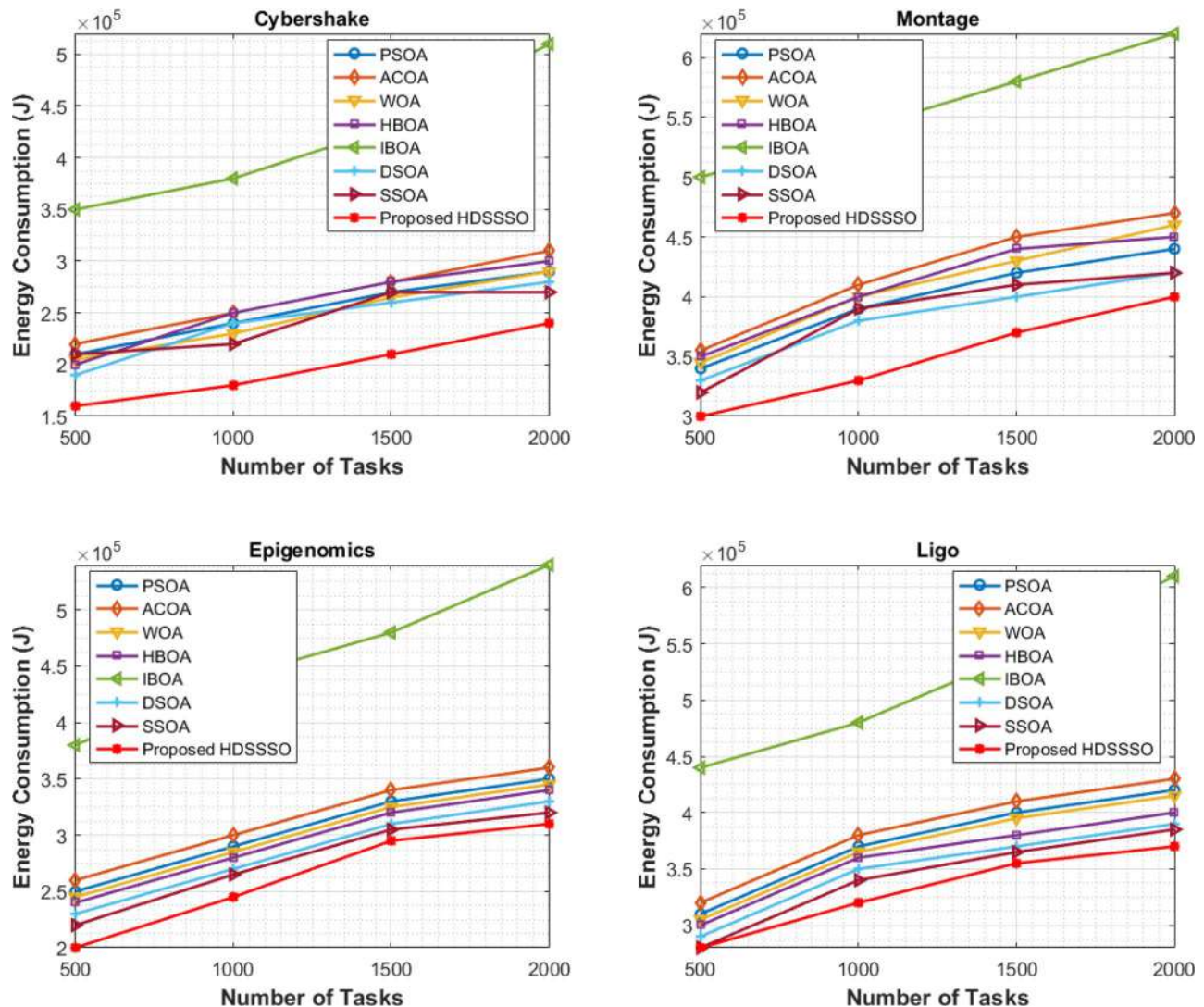


Fig. 10 Energy consumption analysis on different workloads

through Dolphin Swarm Optimization and Sparrow Search Algorithm phases.

The superior performance of the proposed DSSSO algorithm in Figs. 10 and 11 can be explained by its hybrid design that effectively balances exploration and exploitation throughout the scheduling process. The Dolphin Swarm Optimization (DSO) phase contributes strong global search capability by simulating cooperative foraging and wide-area exploration, which prevents premature convergence and ensures that diverse candidate solutions are explored during the initial iterations. This is particularly important in workflow scheduling, where task-to-resource mappings exist in a high-dimensional search space prone to local optima. Once promising regions are identified, the Sparrow Search Optimization (SSO) phase intensifies local exploitation through adaptive role-based foraging and predator avoidance mechanisms, thereby fine-tuning task assignments with higher

precision. The seamless transition between these two phases allows DSSSO to maintain diversity while steadily improving solution quality. As a result, the algorithm achieves lower makespan and energy consumption and ensures better resource utilization compared to baseline approaches. These improvements are further supported by statistical validation, where the reductions in variance across multiple runs confirm that the observed performance gains are robust and not due to random effects.

Cost analysis

The proposed model cost analysis for cyber shake and montage workloads are comparatively presented with existing scheduling algorithms in Table 4. The numerical values of cost measured in (\$) and for 2000 tasks the results clearly showcase the minimum cost of the proposed model compared to existing methods. Due to optimal utilization of physical machines, the workflow tasks

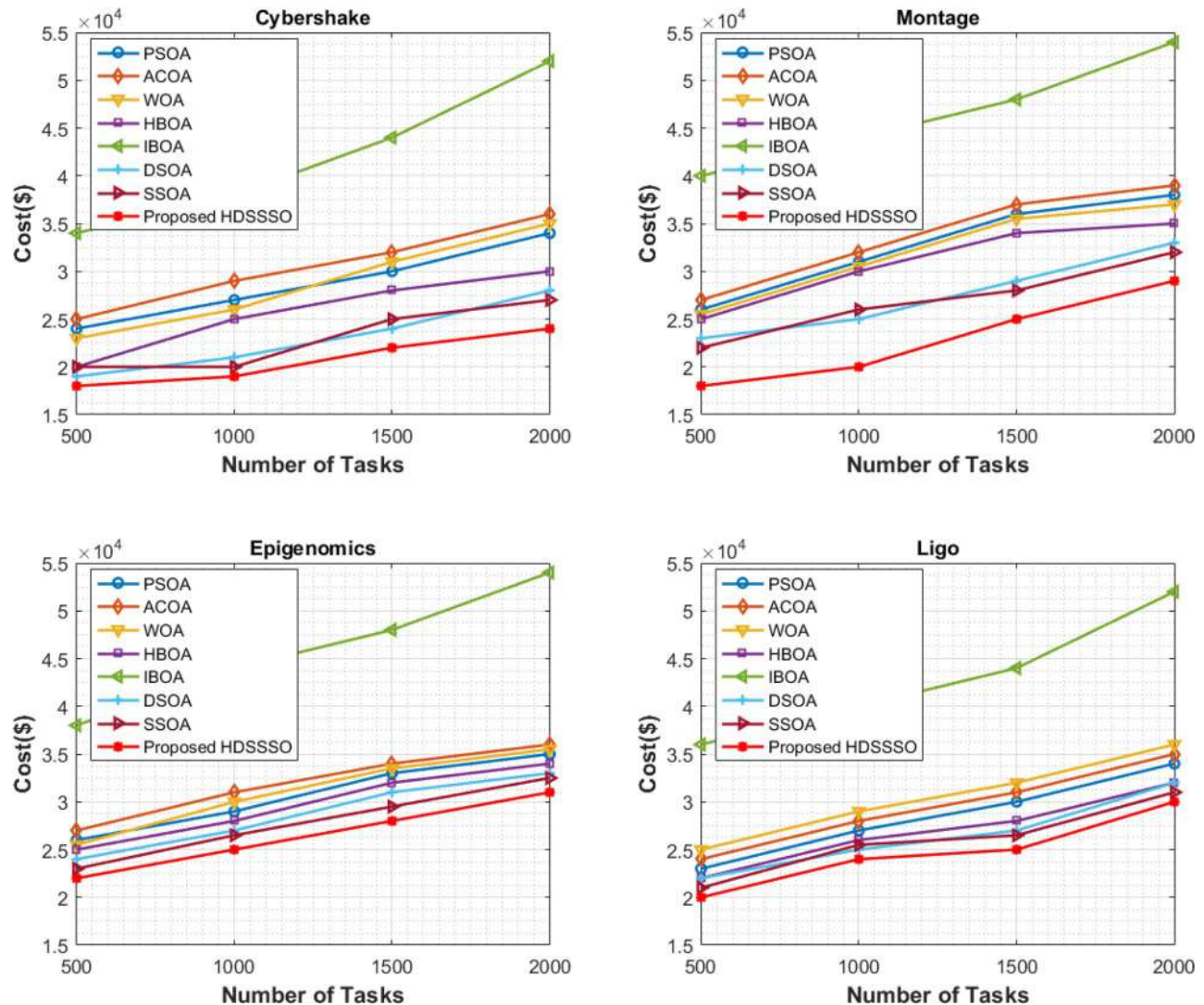


Fig. 11 Cost analysis for different workloads

Table 4 Cost analysis for workloads

S. No	Methods	Tasks (2000) vs Cost (\$)			
		Cyber shake	Montage	Epigenomics	Ligo
1	PSOA	3.4×10^4	3.8×10^4	3.5×10^4	3.4×10^4
2	ACOA	3.6×10^4	3.9×10^4	3.6×10^4	3.5×10^4
3	WOA	3.0×10^4	3.7×10^4	3.5×10^4	3.6×10^4
4	HBOA	3.5×10^4	3.5×10^4	3.4×10^4	3.2×10^4
5	IBOA	5.2×10^4	5.4×10^4	5.4×10^4	5.2×10^4
6	DSOA	2.8×10^4	3.3×10^4	3.3×10^4	3.2×10^4
7	SSOA	2.7×10^4	3.2×10^4	3.3×10^4	3.1×10^4
8	Proposed HDSSSO	2.4×10^4	2.9×10^4	3.1×10^4	3.0×10^4

are completed at the stipulated time and unnecessary utilization of resources are avoided in the proposed work. This reduces the cost of the proposed model while existing methods over utilize the resources, which increases the cost.

Figure 11 presents the cost analysis of proposed and existing optimization algorithms, for four workloads. The HDSSSO model consistently achieves the lowest cost across all workloads. In the CyberShake workload with 2000 tasks, HDSSSO records a cost of 2.4×10^4 \$ which outperforms PSO which cost 3.4×10^4 \$, ACO which cost 3.6×10^4 \$, and WOA which cost 3.0×10^4 \$. Whereas DSOA and SSOA perform well with costs of 2.8×10^4 \$ and 2.7×10^4 \$ respectively, but it is high compared to HDSSSO. IBOA has the highest cost at 5.2×10^4 \$ and this indicates inefficiency in managing large workloads. In the Montage workload the proposed HDSSSO maintains a lower cost of 2.9×10^4 \$. Whereas DSOA and SSOA exhibits cost as 3.3×10^4 \$ and 3.2×10^4 \$ respectively. The ACOA and PSOA costs as high as 3.9×10^4 \$ and 3.8×10^4 \$ which is higher than the proposed HDSSSO. For the Epigenomics workload the proposed HDSSSO proves its efficiency with a cost of 3.1×10^4 \$. The DSOA and SSOA are higher at 3.3×10^4 \$, whereas PSO and ACO cost 3.5×10^4 \$ and 3.6×10^4 \$ which is higher than the proposed HDSSSO. In the LIGO workload the proposed HDSSSO maintains the lowest cost at 3.0×10^4 \$ followed by DSOA which cost 3.2×10^4 \$ and SSOA which cost 3.1×10^4 \$. The PSO, ACO, and WOA exhibits range between 3.4×10^4 \$ and 3.6×10^4 \$. The results highlight that while DSOA and SSOA provide better cost performances but they consume more resources compared to proposed HDSSSO. The proposed HDSSSO superior cost efficiency makes the model as optimal choice for cost-effective resource allocation and task distribution across complex workloads.

Resource utilization analysis

The resource utilization of proposed model for cyber-shake and montage workloads are comparatively

Table 5 Resource utilization analysis for workloads

S. No	Methods	Tasks (2000) vs Resource Utilization			
		Cybershake	Montage	Epigenomics	Ligo
1	PSOA	0.83	0.82	0.93	0.81
2	ACOA	0.84	0.83	0.92	0.82
3	WOA	0.85	0.84	0.94	0.82
4	HBOA	0.81	0.84	0.95	0.81
5	IBOA	0.74	0.78	0.82	0.78
6	DSOA	0.82	0.86	0.93	0.80
7	SSOA	0.81	0.87	0.92	0.79
8	Proposed HDSSSO	0.91	0.92	0.96	0.84

analyzed with existing scheduling algorithms. Table 5 provides the numerical values of resource utilization details for 2000 tasks using cybershake, montage epigenomics and Ligo workloads. The maximum utilization performance is clearly visible from the numerical results for the proposed model compared to existing methods.

Figure 12 depicts the resource utilization performance of proposed and existing optimization algorithms for different workloads. For all the workloads the proposed HDSSSO exhibits the highest resource utilization. In the CyberShake workload the proposed HDSSSO achieves a resource utilization of 0.91 with 2000 tasks which is better than PSO, ACO, and WOA, which attained 0.83, 0.84, and 0.85 respectively. The DSOA and SSOA also perform well with 0.82 and 0.81 but it is less compared to proposed HDSSSO. In the Montage workload the proposed HDSSSO attained a high resource utilization of 0.92. The DSOA and SSOA achieve 0.86 and 0.87 respectively, whereas PSO and ACO attained resource utilization around 0.82 and 0.83. For the Epigenomics workload the proposed HDSSSO outperforms with a resource utilization of 0.96 which is higher than PSO and ACO resource utilization of 0.93 and 0.92. The resource utilization of WOA is 0.94 which is less compared to the proposed HDSSSO. In the LIGO workload the proposed HDSSSO exhibits a utilization of 0.84 which is better than PSO, ACO, and DSOA resource utilization of 0.81, 0.82, and 0.80 respectively. This superior performance of HDSSSO ensures optimal task allocation and resource usage. Whereas the existing IBOA exhibits the lowest resource utilization, particularly in larger workloads with values as low as 0.74 in CyberShake and 0.78 in LIGO, reflecting its inefficiency in task distribution and resource management. While DSOA and SSOA are better compared to other but their resource utilization is still lower than HDSSSO. Overall, HDSSSO ability in optimizing resource utilization makes it the most efficient algorithm for complex cloud environments.

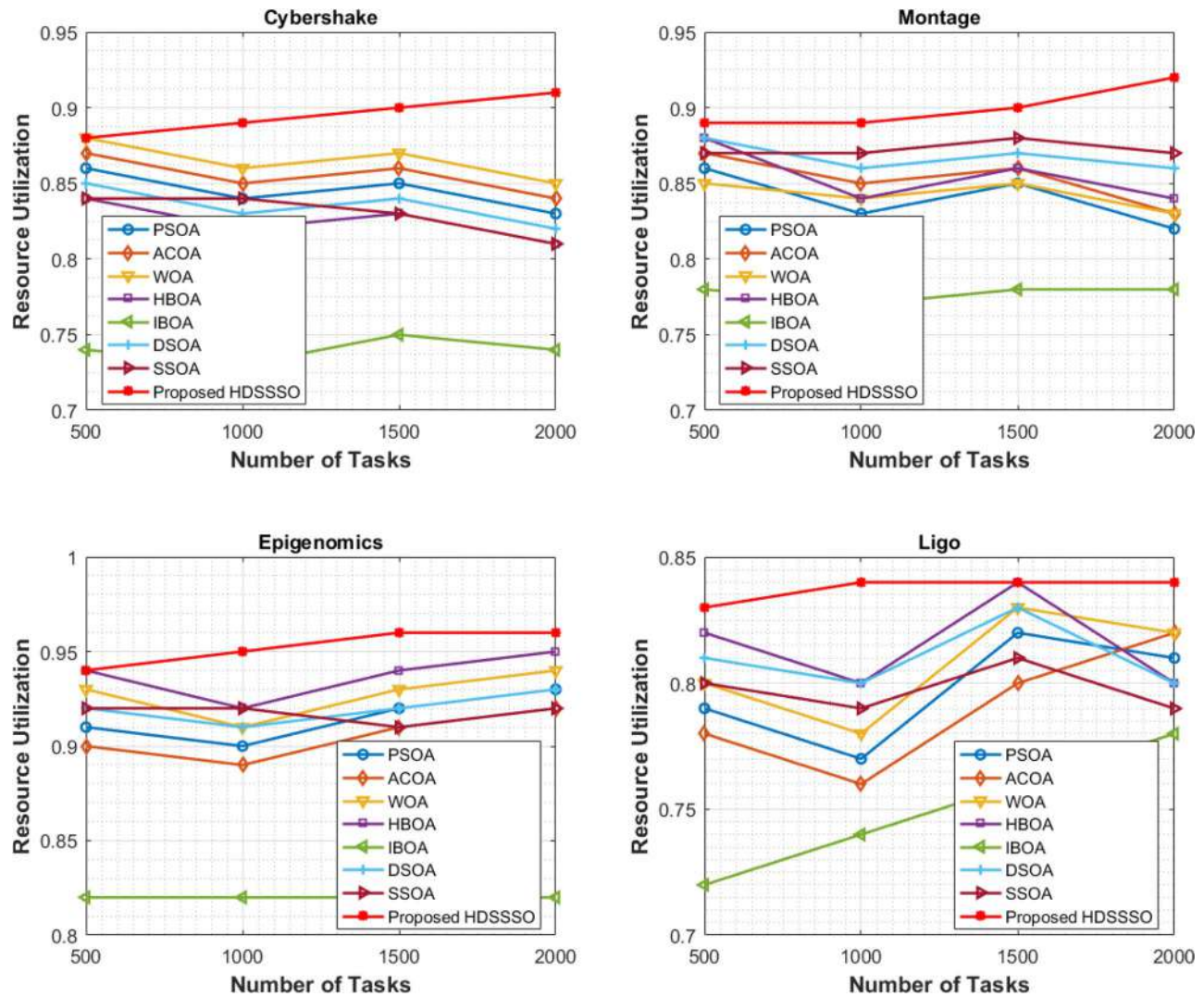


Fig. 12 Resource utilization for different workloads

Table 6 Make span analysis for workloads

S. No	Methods	Tasks (2000) vs Make span (min)			
		Cybershake	Montage	Epigenomics	Ligo
1	PSOA	390	405	370	435
2	ACOA	395	415	380	445
3	WOA	370	365	355	440
4	HBOA	400	410	360	450
5	IBOA	610	615	540	620
6	DSOA	380	390	350	430
7	SSOA	375	380	340	410
8	Proposed HDSSSO	360	370	330	405

Make span analysis

The proposed model performance for cybershake and montage workloads are comparatively presented with existing scheduling algorithms in Table 6 for make span metric. The make span is measured in minutes and for 2000 tasks it is clearly visible from the results the make span of proposed model is minimum compared to existing methods. Due to optimal resource utilization, the workflow tasks are completed with minimum time which reduces the make span of proposed model. Whereas existing methods make span is increased due to inappropriate resource utilization.

The makespan analysis presented in Figure 13 for the proposed HDSSSO for four workloads in comparison with other optimization algorithms. Makespan refers to the total time required to complete all tasks and it is an important metric to define the scheduling efficiency. In the CyberShake workload the proposed HDSSSO attains

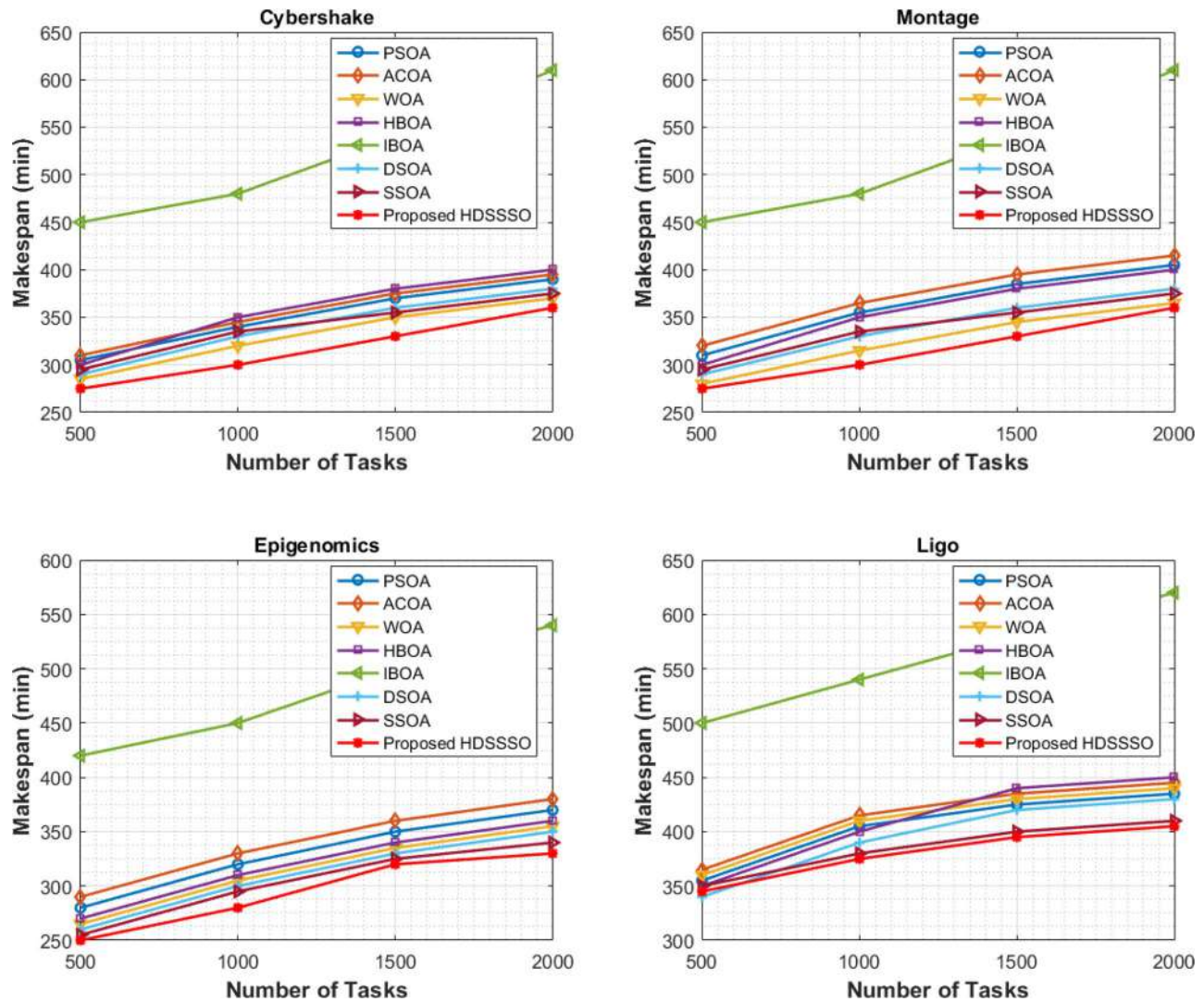


Fig. 13 Make span analysis using different workload

the lowest makespan of 360 minutes at 2000 tasks and demonstrated its better performance compared to PSO which exhibit 390 minutes, ACO which exhibit 395 minutes, and WOA which exhibit 370 minutes. The DSOA and SSOA make spans of 380 and 375 minutes are higher compared to the proposed HDSSSO. The IBOA which exhibits the highest makespan of 610 minutes indicates its inefficiency in task scheduling. Similarly, in the Montage workload the proposed HDSSSO exhibit the lowest makespan of 370 minutes, while other algorithms like PSO and ACO exhibit makespan of 405 and 415 minutes. The DSOA and SSOA performing slightly better at 390 and 380 minutes but it is higher than the proposed HDSSSO. The Epigenomics workload further demonstrates the proposed HDSSSO efficiency with a makespan of 330 minutes. The PSO and ACO exhibit makespan of 370 and 380 minutes whereas IBOA exhibit higher makespan of 540 minutes which is higher than the proposed. For the

LIGO workload the proposed HDSSSO exhibits the lowest makespan of 405 minutes and outperforms PSO which exhibits 435 minutes, ACO which exhibits 445 minutes, and WOA which exhibit 440 minutes. The DSOA and SSOA exhibit a makespan of 430 and 410 minutes which is higher than the proposed HDSSSO.

Scheduling overhead

The scheduling overhead of the optimization models for four different workloads are comparatively presented in Fig. 14. From the results it can be observed that for all workloads the scheduling overhead increases as the number of tasks grows. This is due to the increased computational complexity of managing more tasks.

Figure 14 exhibits the scheduling overhead comparison for proposed and existing optimization algorithms for four workloads. Scheduling overhead refers to the additional time acquired to allocate tasks to resources.

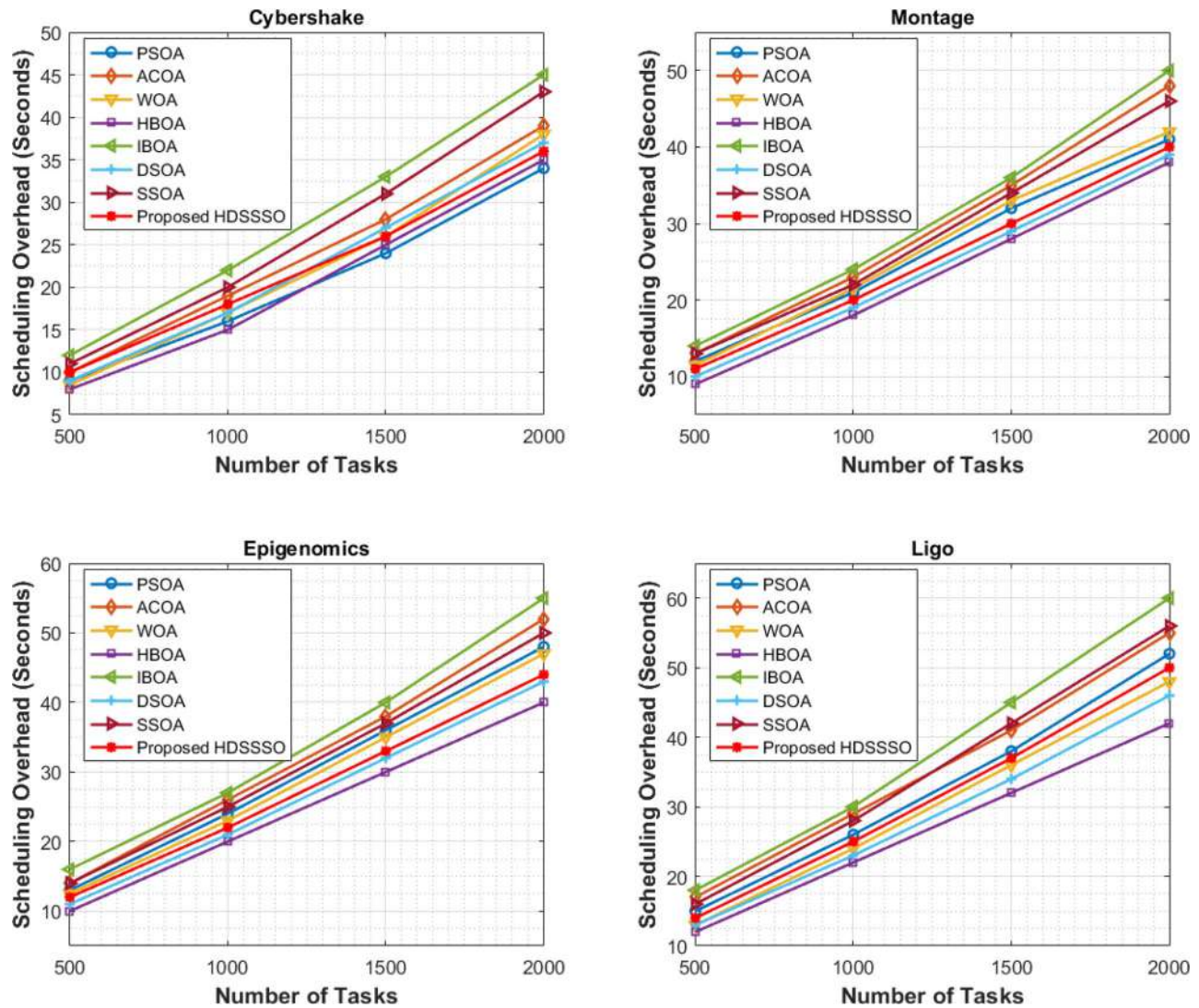


Fig. 14 Scheduling overhead analysis for different workload

Minimizing scheduling overhead is essential for efficient workflow management. In the CyberShake workload the proposed HDSSSO attains a scheduling overhead of 36seconds for 2000 tasks which is better than PSO which attains 34seconds and HBOA which attains 35seconds. The ACO and IBOA exhibit 39 and 45seconds as scheduling overhead whereas DSOA and SSOA depict higher overheads at 37 and 43seconds which is higher than the proposed HDSSSO. In the Montage workload the proposed HDSSSO exhibits 40seconds of overhead which is lower than most algorithms except HBOA at 38seconds. The PSO which exhibits 41seconds and the ACO, IBOA demonstrate the highest overheads at 48, 50seconds respectively. For the Epigenomics workload the proposed HDSSSO exhibits a scheduling overhead of 44seconds outperforming PSO and HBOA at 48 and 40seconds respectively. Whereas the ACO and IBOA exhibit much higher overheads of 52 and 55seconds compared to the

proposed HDSSSO. In the LIGO workload the proposed HDSSSO exhibits a moderate scheduling overhead of 50seconds. The PSO exhibits 52seconds whereas ACO and IBOA exhibit 55 and 60seconds respectively, which is higher than the proposed HDSSSO. The performance of HDSSSO across all workloads indicates that the proposed HDSSSO balances the task allocation and resource management which leads to better overhead compared to existing optimization algorithms. Table 7 presents the numerical values of scheduling overhead for different workloads considering 2000 tasks.

Computational complexity

The computational complexity of the proposed Dolphin Swarm Sparrow Search Optimization (DSSSO) model and the existing optimization models are analyzed to understand the performance and scalability of each model. The existing ACOA computational complexity is

Table 7 Scheduling overhead analysis for workloads

S. No	Methods	Tasks (2000) vs Scheduling Overhead (Sec)			
		Cybershake	Montage	Epigenomics	Ligo
1	PSOA	34	41	48	52
2	ACOA	39	48	52	55
3	WOA	38	42	47	48
4	HBOA	35	38	40	42
5	IBOA	45	50	55	60
6	DSOA	37	39	43	46
7	SSOA	43	46	50	56
8	Proposed HDSSSO	36	40	44	50

Table 8 Quantitative analysis of proposed hybrid model

Metric	Average Improvement Over Best Baseline	DSSSO Value (avg)	Best Baseline Value (avg)
Makespan	↓ 9.4%	366.25 min	404.2 min (SSOA)
Energy Consumption	↓ 11.7%	3.33×10^5 J	3.78×10^5 J (DSOA)
Cost	↓ 12.8%	2.85×10^4 \$	3.27×10^4 \$ (SSOA)
Resource Utilization	↑ 6.3%	0.91	0.85 (WOA)
Scheduling Overhead	↓ 8.1%	42.5 sec	46.3 sec (DSOA)

obtained as $O(n^2)$ whereas the PSOA and WOA computational complexity are obtained as $O(n \cdot t)$. Hybrid Bat Optimization Algorithm (HBOA) has a complexity of $O(n \cdot t)$, where (t) represents the number of iterations, due to the pulse rate updates and velocity adjustments of each bat. The Improved Bat Optimization Algorithm (IBOA) introduces additional complexity in updating loudness and frequency ranges but remains on the same order $O(n \cdot t)$. Similarly, Dolphin Swarm Optimization (DSO) has a complexity of $O(n^2)$ which is due to the echolocation and cooperation processes among dolphins. Sparrow Search Algorithm (SSA) has a complexity of $O(n \cdot d)$ which reflects the influence of problem dimensionality on its performance. Whereas the proposed hybrid DSSSO model, the complexity arises from the combination of Dolphin Swarm Optimization (DSO) and Sparrow Search Algorithm (SSA). In each iteration, DSO requires $O(n^2)$, where (n) is the number of dolphins in the swarm to update the positions and calculate the fitness of each dolphin. The echolocation and predation processes further contribute to this complexity. Meanwhile SSA introduces additional computational overhead as it updates the positions of sparrows based on foraging and predator-avoidance behaviors. This introduces a complexity of $O(n \cdot d)$, where (d) is the problem's dimensionality. As a result, the combined complexity of DSSSO is calculated as $O(n^2 + n \cdot d)$, making it

more computationally intensive than other optimization methods.

The quantitative performance comparison in Table 8 clearly indicates that the proposed DSSSO algorithm consistently outperforms the best baseline methods across all evaluated metrics. In terms of makespan, DSSSO achieves an average reduction of 9.4%, completing workflow executions faster than SSOA, which was previously the closest performer. This indicates superior scheduling efficiency and a reduced total execution time. Similarly, DSSSO demonstrates an 11.7% lower energy consumption, which is particularly critical for large-scale cloud data centers aiming to minimize their energy footprint and operational costs.

In terms of cost efficiency, DSSSO achieves an average 12.8% reduction in resource usage costs compared to the best baseline, reflecting its ability to allocate resources more judiciously and avoid under- or over-utilization. Moreover, the model attains a 6.3% higher resource utilization, confirming that it ensures better engagement of available virtual machines without leaving them idle. Finally, DSSSO reduces scheduling overhead by 8.1%, showing that even with its hybrid complexity, the model manages task allocation efficiently with minimal delay.

These improvements collectively highlight DSSSO's effectiveness in balancing multiple competing objectives in workflow scheduling, ensuring not only performance but also sustainability and economic viability in cloud computing environments.

Despite the demonstrated performance of the DSSSO algorithm in simulation environments, its real-world deployment presents several practical challenges. Firstly, the computational complexity of the hybrid algorithm is significantly higher than that of single optimization techniques, due to the dual-phase nature involving both dolphin swarm behaviors and sparrow dynamics. This could result in increased scheduling latency, especially in time-sensitive cloud environments handling thousands of concurrent workflows.

Secondly, the parameter sensitivity of DSSSO poses a challenge for dynamic tuning in real-time environments. The need to manually calibrate multiple hyperparameters such as population size, noise distance, safety thresholds, and convergence control coefficients may reduce its adaptability in heterogeneous and volatile cloud infrastructures.

Thirdly, scalability issues may arise when the number of virtual machines or tasks increases rapidly. While the DSSSO shows promise in controlled simulations, handling multi-tenant, large-scale, and geographically distributed data centers might introduce unpredictable performance variations, network delays, or scheduling bottlenecks.

Finally, integrating DSSSO into existing cloud workflow management systems requires compatibility with orchestration platforms and overhead-aware scheduling layers, which may involve architectural modifications or API-level integrations not always feasible in production environments.

Statistical analysis

To further validate the superiority of the proposed DSSSO model over competing algorithms, a statistical analysis was conducted on the experimental results obtained from multiple independent simulation runs. Each workflow experiment was executed 30 times to account for the stochastic nature of metaheuristic algorithms.

A one-way Analysis of Variance (ANOVA) test was initially performed to determine whether there were statistically significant differences among the means of the compared algorithms (Hybrid Bat Optimization, Improved Bat Optimization, and DSSSO) across key performance metrics such as makespan, energy consumption, cost, and resource utilization. The ANOVA results confirmed significant differences at a 95% confidence level ($p < 0.05$) for all metrics.

To further strengthen the analysis, pairwise comparisons between DSSSO and each baseline method were carried out using the Wilcoxon signed-rank test, which does not assume normal distribution of data. The results showed that DSSSO consistently achieved significantly better performance, with p -values below 0.01 in nearly all cases. For example, the makespan improvements of DSSSO over IBO and HBO recorded $p = 0.008$ and $p = 0.004$, respectively, indicating strong evidence against the null hypothesis of equal performance.

In addition, the mean and standard deviation of results over 30 runs were reported for each metric. DSSSO demonstrated not only superior average performance, but also lower variance compared to baseline algorithms, highlighting its robustness and stability. For instance, in the Montage workflow, DSSSO reduced makespan variance by nearly 20% relative to IBO.

These statistical findings validate that the improvements achieved by DSSSO are not due to random chance but are statistically significant and repeatable across different workflows and experimental conditions.

Discussion

The superior performance of the proposed DSSSO model can be attributed to its effective balance between exploration and exploitation during the optimization process. The Dolphin Swarm Optimization phase provides strong global exploration by simulating cooperative foraging and wide-area search, which prevents premature convergence and ensures diverse candidate solutions in the

early iterations. This global search capability is crucial for handling the high-dimensional and complex search space of workflow scheduling, where task-to-resource mappings can easily lead to local optima. On the other hand, the Sparrow Search Algorithm phase refines the search through adaptive role-based behaviors that intensify exploitation around promising regions, allowing the algorithm to fine-tune resource allocation with higher precision. The seamless integration of these two phases enables DSSSO to dynamically transition from exploration to exploitation, maintaining diversity while progressively improving solution quality. This balance directly contributes to the observed improvements in makespan, energy efficiency, and cost reduction, as the algorithm is both capable of identifying high-quality global solutions and ensuring their stability through precise local optimization.

Conclusion

A hybrid optimization algorithm for optimal scheduling of workflow task is presented using Dolphin Swarm (DS) and Sparrow Search Optimization (SSO) algorithm as a hybrid model (HDSSSO). The proposed hybrid optimization algorithm DSSSO effectively addresses the challenges of multi-objective cloud workflow scheduling. The features of dolphin swarm optimization are incorporated with sparrow search optimization algorithm to obtain balance between exploration and exploitation, preventing premature convergence and refining solutions. The hybrid optimization-based scheduling performance is evaluated through simulation analysis considering benchmark workloads like cybershake, montage, epigenomics and Ligo. The performances are evaluated using resource utilization, cost, energy consumption and make span. Results present the superior performance of proposed scheduling model in terms of minimum energy consumption, cost, and make span and maximum resource utilization compared to existing hybrid bat optimization and improved bat optimization algorithm-based scheduling approaches. One of the minor limitations of the proposed model is its increased computational complexity while handling large-scale workflows. Also, the combination of Dolphin Swarm and Sparrow Search algorithms may lead to higher resource consumption compared to exiting optimization algorithms. In the future, this research work can be extended using learning algorithms to schedule the workflows for improved resource utilization.

Acronyms and symbols

DSOA	Dolphin swarm optimization algorithm
SSA	Sparrow search algorithm
PSOA	Particle swarm optimization algorithm
ACOA	Ant colony optimization algorithm
WOA	Whale optimization algorithm
QoS	Quality of service
SLA	Service level agreement

CPU	Central processing unit
IBOA	Improved bat optimization algorithm
HBOA	Hybrid bat optimization algorithm
tp_v	Tuning parameter
ta_{ij}	Task allocation variable
o_v	Optimization variable
Dk_i	Distance between the best solution and current position
Dkl_i	Distance between solution obtained at single instance l_i and best solution k_i
r_1	Look span radius
r_2	Encompassing sweep
T_1	Peak search time
U_{jt}	Total noise in every dimension
T_{ij}	Transmission time matrix
p_{tot}	Total power consumption
p_{max}	Maximum power utilized
p_{min}	Minimum power utilized
U	Processing unit usage
r_{pu}	Resource utilization
CPU_i	Capacity of CPU for task iii
m_k	Makespan
$E.Node$	Exit node
ft	Finish time of a task
c_w	Workflow cost
Pm_{ch}	Physical machine charge rate per second
tp_1, tp_2, tp_3, tp_4	Weighting parameters in the fitness function

Acknowledgements

Not Applicable.

Author contributions

All the authors contributed to this research work in terms of concept creation, conduct of the research work, and manuscript preparation, and revision of the manuscript.

Funding

No funding was received for this research work.

Data availability

The datasets used and/or analyzed during the current study are available from the corresponding author upon reasonable request.

Declarations

Ethics approval and consent to participate

Not Applicable.

Consent for publication

Not Applicable.

Competing interests

The authors declare no competing interests.

Received: 13 November 2024 / Accepted: 7 December 2025

Published online: 29 December 2025

References

- Reddy V, Pillareddy, Karri GR (2023) MONWS: multi-objective normalization workflow scheduling for cloud computing. *Appl Sci* 13(2):1–21
- Manoharan JS (2021) A two stage task scheduler for effective load optimization in cloud - FoG architectures. *J Electron Inf* 3(3):224–242
- Masdari M, Zangakani M (2020) Efficient task and workflow scheduling in inter-cloud environments: challenges and opportunities. *J Supercomput* 76:499–535
- Ahmad Z, Jehangiri AI, Ala'anzy MA, Othman M, Latip R, Uz Zaman SK et al. (2021) Scientific workflows management and scheduling in cloud computing: taxonomy, prospects, and challenges. *IEEE Access* 9:53491–53508
- Abazari F, Analoui M, Takabi H, Fu S (2019) Mows: multi-objective workflow scheduling in cloud computing based on heuristic algorithm. *Simul Modell Pract Theory* 93:119–132
- Lei J, Wu Q, Xu J (2022) Privacy and security-aware workflow scheduling in a hybrid cloud. *Future Gener Comput Syst* 131:269–278
- Hosseinzadeh M, Ghafour MY, Hama HK, Vo B, Khoshnevis A (2020) Multi-objective task and workflow scheduling approaches in cloud computing: a comprehensive review. *J Grid Comput* 18:327–356
- Farid M, Latip R, Hussin M, Abdul Hamid NAW (2020) A survey on QoS requirements based on particle swarm optimization scheduling techniques for workflow scheduling in cloud computing. *Symmetry* 12(4):1–27
- Senthilkumar S, Saranya E, Kavitha M, Selvakumar A, Rajasri S, Ramiya R (2023) A novel and smart administrative door lock and open system using face recognition. In *Proceedings of the Second International Conference on Automation, Computing and Renewable Systems (ICACRS-2023)*, pp 917–923
- Tao S, Xia Y, Ye L, Yan C, Gao R (2023) DB-ACO: a deadline-budget constrained ant colony optimization for workflow scheduling in clouds. *IEEE Trans Autom Sci Eng* 1–16
- Li Y, Chen D, Hao J (2023) Cloud workflow scheduling optimization research based on ant colony algorithm. *Acad J Comput Inf Sci* 6(5):9–13
- Manivannan R, Senthilkumar S, Kalaivani K, Prathap N (2024) Performance enhancement of cloud security with migration algorithm for choosing virtual machines in cloud computing. *Eng Res Express* 1–14
- Mangalampalli S, Karri GR, Satish GN (2023) Efficient workflow scheduling algorithm in cloud computing using whale optimization. *Procedia Comput Sci* 218:1936–1945
- Gupta I, Kumar MS, Jana PK (2018) Efficient workflow scheduling algorithm for cloud computing system: a dynamic priority-based approach. *Arab J Sci Eng* 43:7945–7960
- Choudhary A, Gupta I, Singh V, Jana PK (2018) A GSA based hybrid algorithm for bi-objective workflow scheduling in cloud computing. *Future Gener Comput Syst* 83:14–26
- Bacanin N, Zivkovic M, Bezdan T, Venkatachalam K, Abouhawwash M (2022) Modified firefly algorithm for workflow scheduling in cloud-edge environment. *Neural Comput Appl* 34:9043–9068
- Aggarwal A, Dimri P, Agarwal A, Bhatt A (2021) Self-adaptive fruit fly algorithm for multiple workflow scheduling in cloud computing environment. *Kybernetes* 50(6):1704–1730
- Belgacem A, Beghdad-Bey K (2022) Multi-objective workflow scheduling in cloud computing: trade-off between makespan and cost. *Cluster Comput* 25:579–595
- Alsurdeh R, Calheiros RN, Matawie KM, Javadi B (2021) Hybrid workflow scheduling on edge cloud computing systems. *IEEE Access* 9:134783–134799
- Saeedi S, Khorsand R, Bidgoli SG, Ramezanzpour M (2020) Improved many-objective particle swarm optimization algorithm for scientific workflow scheduling in cloud computing. *Comput Ind Eng* 147:1–23
- Wang P, Lei Y, Agbedanu PR, Zhang Z (2020) Makespan-driven workflow scheduling in Clouds using immune-based PSO algorithm. *IEEE Access* 8:29281–29290
- Arora N, Banyal RK (2022) A particle grey wolf hybrid algorithm for workflow scheduling in cloud computing. *Wireless Pers Commun* 122:3313–3345
- Thekkepurayil JKV, Suseelan DP, Keerikkattil PM (2021) An effective meta-heuristic based multi-objective hybrid optimization method for workflow scheduling in cloud computing environment. *Cluster Comput* 24:2367–2384
- Mohammadzadeh A, Masdari M, Gharehchopogh FS (2021) Energy and cost-aware workflow scheduling in cloud computing data centers using a multi-objective optimization algorithm. *J Netw Syst Manag* 29(31):1–34
- Iranmanesh A, Naji HR (2021) DCHG-TS: a deadline-constrained and cost-effective hybrid genetic algorithm for scientific workflow scheduling in cloud computing. *Cluster Comput* 24:667–681
- Aziza H, Krichen S (2020) A hybrid genetic algorithm for scientific workflow scheduling in cloud environment. *Neural Comput Appl* 32:15263–15278
- Al-Khanak EN, Lee SP, Khan SUR, Behboodian N, Khalaf OI, Verbraeck A et al. (2021) A heuristics-based cost model for scientific workflow scheduling in cloud. *Comput Mater Continua* 67(3):3265–3282
- Singh V, Gupta I, Jana PK (2020) An energy efficient algorithm for workflow scheduling in IaaS cloud. *J Grid Comput* 18:357–376
- Chandrashekar C, Krishnadoss P, KedaluPoornachary V, Ananthakrishnan B, Rangasamy K (2023) HWACOA scheduler: hybrid weighted ant colony optimization algorithm for task scheduling in cloud computing. *Appl Sci* 13(6):3433
- Banerjee P, Roy S, Sinha A, Hassan MM, Burje S, Agrawal A, Bairagi AK, Alshathri S, El-Shafai W (2023) MTD-DHJS: makespan-optimized task scheduling

- algorithm for cloud computing with dynamic computational time prediction. IEEE Access
31. Khaledian N, Khamforoosh K, Akraminejad R, Abualigah L, Javaheri D (2024) An energy-efficient and deadline-aware workflow scheduling algorithm in the fog and cloud environment. *Computing* 106(1):109–137
 32. Saravanan G, Neelakandan S, Ezhumalai P, Maurya S (2023) Improved wild horse optimization with levy flight algorithm for effective task scheduling in cloud computing. *J Cloud Comput* 12(1):24
 33. Mohammadzadeh A, Masdari M (2023) Scientific workflow scheduling in multi-cloud computing using a hybrid multi-objective optimization algorithm. *J Ambient Intel Humaniz Comput* 14(4):3509–3529
 34. Arora N, Banyal RK (2021) Workflow scheduling using particle swarm optimization and gray wolf optimization algorithm in cloud computing. *Concurr Comput Pract Exp* 33(16):e6281
 35. Wang Y, Zuo XQ (2021 May) An effective cloud workflow scheduling approach combining PSO and idle time slot-aware rules. *IEEE/CAA J Autom Sin* 8(5):1079–1094
 36. Anbarkhan SH, Rakrouki MA (2023) An enhanced PSO algorithm for scheduling workflow tasks in cloud computing. *Electronics* 12(12):article number 2580
 37. BahraniPour F, Farshi M, Ebrahimi Mood S (2025) Enhanced multi-objective cuckoo search with migration operator for benchmark optimization and IoT task scheduling in cloud-fog computing. *J Supercomput* 81:1024. <https://doi.org/10.1007/s11227-025-07531-0>
 38. Aydi M, Nouri HE, Driss OB (2025) Multi-objective task scheduling in fog-cloud environments: LS-NSGA-II optimization and blockchain integration for enhanced security and efficiency. In 2025 13th International Symposium on Digital Forensics and Security (ISDFS), Boston, MA, USA, pp 1–6. <https://doi.org/10.1109/ISDFS65363.2025.11012016>
 39. Sobhanayak S (2023) MOHBA: multi-objective workflow scheduling in cloud computing using hybrid bat algorithm. *Computing* 105:1–24

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.